

An object, or a set of objects, can be retrieved based on a selection condition on its attributes; relevant data are retrieved from the underlying database based on the selection conditions, and one or more objects are created from the retrieved data, based on the prespecified mapping between objects and relations.

The program can update retrieved objects, create new objects, or specify that an object is to be deleted, and then issue a save command; the mapping from objects to relations is then used to correspondingly update, insert, or delete tuples in the database.

The primary goal of object-relational mapping systems is to ease the job of programmers who build applications by providing them an object model while retaining the benefits of using a robust relational database underneath. As an added benefit, when operating on objects cached in memory, object-relational systems can provide significant performance gains over direct access to the underlying database.

Object-relational mapping systems also provide query languages that allow programmers to write queries directly on the object model; such queries are translated into SQL queries on the underlying relational database, and result objects are created from the SQL query results.

A fringe benefit of using an ORM is that any of a number of databases can be used to store data, with exactly the same high-level code. ORMs hide minor SQL differences between databases from the higher levels. Migration from one database to another is thus relatively straightforward when using an ORM, whereas SQL differences can make such migration significantly harder if an application uses SQL to communicate with the database.

On the negative side, object-relational mapping systems can suffer from significant performance inefficiencies for bulk database updates, as well as for complex queries that are written directly in the imperative language. It is possible to update the database directly, bypassing the object-relational mapping system, and to write complex queries directly in SQL in cases where such inefficiencies are discovered.

The benefits of object-relational models exceed the drawbacks for many applications, and object-relational mapping systems have seen widespread adoption in recent years. In particular, Hibernate has seen wide adoption with Java, while several ORMs including Django and SQLAlchemy are widely used with Python. More information on the Hibernate ORM system, which provides an object-relational mapping for Java, and the Django ORM system, which provides an object-relational mapping for Python, can be found in Section 9.6.2.

8.3 Textual Data

Textual data consists of unstructured text. The term **information retrieval** generally refers to the querying of unstructured textual data. In the traditional model used in the field of information retrieval, textual information is organized into *documents*. In a database, a text-valued attribute can be considered a document. In the context of the web, each web page can be considered to be a document.

8.3.1 Keyword Queries

Information retrieval systems support the ability to retrieve documents with some desired information. The desired documents are typically described by a set of **keywords**—for example, the keywords “database system” may be used to locate documents on database systems, and the keywords “stock” and “scandal” may be used to locate articles about stock-market scandals. Documents have associated with them a set of keywords; typically, all the words in the documents are considered keywords. A **keyword query** retrieves documents whose set of keywords contains all the keywords in the query.

In its simplest form, an information-retrieval system locates and returns all documents that contain all the keywords in the query. More-sophisticated systems estimate the relevance of documents to a query so that the documents can be shown in order of estimated relevance. They use information about keyword occurrences, as well as hyperlink information, to estimate relevance.

Keyword search was originally targeted at document repositories within organizations or domain-specific document repositories such as research publications. But information retrieval is also important for documents stored in a database.

Keyword-based information retrieval can be used not only for retrieving textual data, but also for retrieving other types of data, such as video and audio data, that have descriptive keywords associated with them. For instance, a video movie may have associated with it keywords such as its title, director, actors, and genre, while an image or video clip may have tags, which are keywords describing the image or video clip, associated with it.

Web search engines are, at core, information retrieval systems. They retrieve and store web pages by *crawling* the web. Users submit keyword queries, and the information retrieval part of the web search engine finds stored web pages containing the required keyword. Web search engines have today evolved beyond just retrieving web pages. Today, search engines aim to satisfy a user’s information needs by judging what topic a query is about and displaying not only web pages judged as relevant but also other kinds of information about the topic. For example, given a query term “cricket”, a search engine may display scores from ongoing or recent cricket matches, rather than just top-ranked documents related to cricket. As another example, in response to a query “New York”, a search engine may show a map of New York and images of New York in addition to web pages related to New York.

8.3.2 Relevance Ranking

The set of all documents that contain the keywords in a query may be very large; in particular, there are billions of documents on the web, and most keyword queries on a web search engine find hundreds of thousands of documents containing some or all of the keywords. Not all the documents are equally relevant to a keyword query. Information-retrieval systems therefore estimate relevance of documents to a query and return only highly ranked documents as answers. Relevance ranking is not an exact science, but there are some well-accepted approaches.

8.3.2.1 Ranking Using TF-IDF

The word **term** refers to a keyword occurring in a document, or given as part of a query. The first question to address is, given a particular term t , how relevant is a particular document d to the term. One approach is to use the number of occurrences of the term in the document as a measure of its relevance, on the assumption that more relevant terms are likely to be mentioned many times in a document. Just counting the number of occurrences of a term is usually not a good indicator: first, the number of occurrences depends on the length of the document, and second, a document containing 10 occurrences of a term may not be 10 times as relevant as a document containing one occurrence.

One way of measuring $TF(d, t)$, the relevance of a term t to a document d , is:

$$TF(d, t) = \log \left(1 + \frac{n(d, t)}{n(d)} \right)$$

where $n(d)$ denotes the number of term occurrences in the document and $n(d, t)$ denotes the number of occurrences of term t in the document d . Observe that this metric takes the length of the document into account. The relevance grows with more occurrences of a term in the document, although it is not directly proportional to the number of occurrences.

Many systems refine the above metric by using other information. For instance, if the term occurs in the title, or the author list, or the abstract, the document would be considered more relevant to the term. Similarly, if the first occurrence of a term is late in the document, the document may be considered less relevant than if the first occurrence is early in the document. The above notions can be formalized by extensions of the formula we have shown for $TF(d, t)$. In the information retrieval community, the relevance of a document to a term is referred to as **term frequency (TF)**, regardless of the exact formula used.

A query Q may contain multiple keywords. The relevance of a document to a query with two or more keywords is estimated by combining the relevance measures of the document for each keyword. A simple way of combining the measures is to add them up. However, not all terms used as keywords are equal. Suppose a query uses two terms, one of which occurs frequently, such as “database”, and another that is less frequent, such as “Silberschatz”. A document containing “Silberschatz” but not “database” should be ranked higher than a document containing the term “database” but not “Silberschatz”.

To fix this problem, weights are assigned to terms using the **inverse document frequency (IDF)**, defined as:

$$IDF(t) = \frac{1}{n(t)}$$

where $n(t)$ denotes the number of documents (among those indexed by the system) that contain the term t . The **relevance** of a document d to a set of terms Q is then defined as:

$$r(d, Q) = \sum_{t \in Q} TF(d, t) * IDF(t)$$

This measure can be further refined if the user is permitted to specify weights $w(t)$ for terms in the query, in which case the user-specified weights are also taken into account by multiplying $TF(t)$ by $w(t)$ in the preceding formula.

The above approach of using term frequency and inverse document frequency as a measure of the relevance of a document is called the **TF-IDF** approach.

Almost all text documents (in English) contain words such as “and,” “or,” “a,” and so on, and hence these words are useless for querying purposes since their inverse document frequency is extremely low. Information-retrieval systems define a set of words, called **stop words**, containing 100 or so of the most common words, and ignore these words when indexing a document. Such words are not used as keywords, and they are discarded if present in the keywords supplied by the user.

Another factor taken into account when a query contains multiple terms is the **proximity** of the terms in the document. If the terms occur close to each other in the document, the document will be ranked higher than if they occur far apart. The formula for $r(d, Q)$ can be modified to take proximity of the terms into account.

Given a query Q , the job of an information-retrieval system is to return documents in descending order of their relevance to Q . Since there may be a very large number of documents that are relevant, information-retrieval systems typically return only the first few documents with the highest degree of estimated relevance and permit users to interactively request further documents.

8.3.2.2 Ranking Using Hyperlinks

Hyperlinks between documents can be used to decide on the overall importance of a document, independent of the keyword query; for example, documents linked from many other documents are considered more important.

The web search engine Google introduced **PageRank**, which is a measure of popularity of a page based on the popularity of pages that link to the page. Using the PageRank popularity measure to rank answers to a query gave results so much better than previously used ranking techniques that Google became the most widely used search engine in a rather short period of time.

Note that pages that are pointed to from many web pages are more likely to be visited, and thus should have a higher PageRank. Similarly, pages pointed to by web pages with a high PageRank will also have a higher probability of being visited, and thus should have a higher PageRank.

The PageRank of a document d is thus defined (circularly) based on the PageRank of other documents that link to document d . PageRank can be defined by a set of linear equations, as follows: First, web pages are given integer identifiers. The jump probability matrix T is defined with $T[i, j]$ set to the probability that a random walker who is following a link out of page i follows the link to page j . Assuming that each link

from i has an equal probability of being followed $T[i, j] = 1/N_i$, where N_i is the number of links out of page i . Then the PageRank $P[j]$ for each page j can be defined as:

$$P[j] = \delta/N + (1 - \delta) * \sum_{i=1}^N (T[i, j] * P[i])$$

where δ is a constant between 0 and 1, usually set to 0.15, and N is the number of pages.

The set of equations generated as above are usually solved by an iterative technique, starting with each $P[i]$ set to $1/N$. Each step of the iteration computes new values for each $P[i]$ using the P values from the previous iteration. Iteration stops when the maximum change in any $P[i]$ value in an iteration goes below some cutoff value.

Note that PageRank is a static measure, independent of the keyword query; given a keyword query, it is used in combination with TF-IDF scores of a document to judge its relevance of the document to the keyword query.

PageRank is not the only measure of the popularity of a site. Information about how often a site is visited is another useful measure of popularity. Further, search engines track what fraction of times users click on a page when it is returned as an answer. Keywords that occur in the anchor text associated with the hyperlink to a page are viewed as very important and are given a higher term frequency. These and a number of other factors are used to rank answers to a keyword query.

8.3.3 Measuring Retrieval Effectiveness

Ranking of results of a keyword query is not an exact science. Two metrics are used to measure how well an information-retrieval system is able to answer queries. The first, **precision**, measures what percentage of the retrieved documents are actually relevant to the query. The second, **recall**, measures what percentage of the documents relevant to the query were retrieved. Since search engines find a very large number of answers, and users typically stop after browsing some number (say, 10 or 20) of the answers, the precision and recall numbers are usually measured “@K”, where K is the number of answers viewed. Thus, one can talk of precision@10 or recall@20.

8.3.4 Keyword Querying on Structured Data and Knowledge Graphs

Although querying on structured data are typically done using query languages such as SQL, users who are not familiar with the schema or the query language find it difficult to get information from such data. Based on the success of keyword querying in the context of information retrieval from the web, techniques have been developed to support keyword queries on structured and semi-structured data.

One approach is to represent the data using graphs, and then perform keyword queries on the graphs. For example, tuples can be treated as nodes in the graph, and foreign key and other connections between tuples can be treated as edges in the graph. Keyword search is then modeled as finding tuples containing the given keywords and finding connecting paths between them in the corresponding graph.

CHAPTER 14



Indexing

Many queries reference only a small proportion of the records in a file. For example, a query like “Find all instructors in the Physics department” or “Find the total number of credits earned by the student with *ID* 22201” references only a fraction of the *instructor* or *student* records. It is inefficient for the system to read every tuple in the *instructor* relation to check if the *dept_name* value is “Physics”. Likewise, it is inefficient to read the entire *student* relation just to find the one tuple for the *ID* “22201”. Ideally, the system should be able to locate these records directly. To allow these forms of access, we design additional structures that we associate with files.

14.1 Basic Concepts

An index for a file in a database system works in much the same way as the index in this textbook. If we want to learn about a particular topic (specified by a word or a phrase) in this textbook, we can search for the topic in the index at the back of the book, find the pages where it occurs, and then read the pages to find the information for which we are looking. The words in the index are in sorted order, making it easy to find the word we want. Moreover, the index is much smaller than the book, further reducing the effort needed.

Database-system indices play the same role as book indices in libraries. For example, to retrieve a *student* record given an *ID*, the database system would look up an index to find on which disk block¹ the corresponding record resides, and then fetch the disk block, to get the appropriate *student* record.

Indices are critical for efficient processing of queries in databases. Without indices, every query would end up reading the entire contents of every relation that it uses; doing so would be unreasonably expensive for queries that only fetch a few records, for example, a single *student* record, or the records in the *takes* relation corresponding to a single student.

¹ As in earlier chapters, we use the term *disk* to refer to persistent storage devices, such as magnetic disks and solid-state drives.

Implementing an index on the *student* relation by keeping a sorted list of students' *ID* would not work well on very large databases, since (i) the index would itself be very big, (ii) even though keeping the index sorted reduces the search time, finding a student can still be rather time-consuming, and (iii) updating a sorted list as students are added or removed from the database can be very expensive. Instead, more sophisticated indexing techniques are used in database systems. We shall discuss several of these techniques in this chapter.

There are two basic kinds of indices:

- **Ordered indices.** Based on a sorted ordering of the values.
- **Hash indices.** Based on a uniform distribution of values across a range of buckets. The bucket to which a value is assigned is determined by a function, called a *hash function*.

We shall consider several techniques for ordered indexing. No one technique is the best. Rather, each technique is best suited to particular database applications. Each technique must be evaluated on the basis of these factors:

- **Access types:** The types of access that are supported efficiently. Access types can include finding records with a specified attribute value and finding records whose attribute values fall in a specified range.
- **Access time:** The time it takes to find a particular data item, or set of items, using the technique in question.
- **Insertion time:** The time it takes to insert a new data item. This value includes the time it takes to find the correct place to insert the new data item, as well as the time it takes to update the index structure.
- **Deletion time:** The time it takes to delete a data item. This value includes the time it takes to find the item to be deleted, as well as the time it takes to update the index structure.
- **Space overhead:** The additional space occupied by an index structure. Provided that the amount of additional space is moderate, it is usually worthwhile to sacrifice the space to achieve improved performance.

We often want to have more than one index for a file. For example, we may wish to search for a book by author, by subject, or by title.

An attribute or set of attributes used to look up records in a file is called a **search key**. Note that this definition of *key* differs from that used in *primary key*, *candidate key*, and *superkey*. This duplicate meaning for *key* is (unfortunately) well established in practice. Using our notion of a search key, we see that if there are several indices on a file, there are several search keys.

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	



Figure 14.1 Sequential file for *instructor* records.

14.2 Ordered Indices

To gain fast random access to records in a file, we can use an index structure. Each index structure is associated with a particular search key. Just like the index of a book or a library catalog, an **ordered index** stores the values of the search keys in sorted order and associates with each search key the records that contain it.

The records in the indexed file may themselves be stored in some sorted order, just as books in a library are stored according to some attribute such as the Dewey decimal number. A file may have several indices, on different search keys. If the file containing the records is sequentially ordered, a **clustering index** is an index whose search key also defines the sequential order of the file. Clustering indices are also called **primary indices**; the term *primary index* may appear to denote an index on a primary key, but such indices can in fact be built on any search key. The search key of a clustering index is often the primary key, although that is not necessarily so. Indices whose search key specifies an order different from the sequential order of the file are called **nonclustering indices**, or **secondary indices**. The terms “clustered” and “nonclustered” are often used in place of “clustering” and “nonclustering.”

In Section 14.2.1 through Section 14.2.3, we assume that all files are ordered sequentially on some search key. Such files, with a clustering index on the search key, are called **index-sequential files**. They represent one of the oldest index schemes used in database systems. They are designed for applications that require both sequential processing of the entire file and random access to individual records. In Section 14.2.4 we cover secondary indices.

Figure 14.1 shows a sequential file of *instructor* records taken from our university example. In the example of Figure 14.1, the records are stored in sorted order of instructor *ID*, which is used as the search key.

14.2.1 Dense and Sparse Indices

An **index entry**, or **index record**, consists of a search-key value and pointers to one or more records with that value as their search-key value. The pointer to a record consists of the identifier of a disk block and an offset within the disk block to identify the record within the block.

There are two types of ordered indices that we can use:

- **Dense index:** In a dense index, an index entry appears for every search-key value in the file. In a dense clustering index, the index record contains the search-key value and a pointer to the first data record with that search-key value. The rest of the records with the same search-key value would be stored sequentially after the first record, since, because the index is a clustering one, records are sorted on the same search key.

In a dense nonclustering index, the index must store a list of pointers to all records with the same search-key value.

- **Sparse index:** In a sparse index, an index entry appears for only some of the search-key values. Sparse indices can be used only if the relation is stored in sorted order of the search key; that is, if the index is a clustering index. As is true in dense indices, each index entry contains a search-key value and a pointer to the first data record with that search-key value. To locate a record, we find the index entry with the largest search-key value that is less than or equal to the search-key value for which we are looking. We start at the record pointed to by that index entry and follow the pointers in the file until we find the desired record.

Figure 14.2 and Figure 14.3 show dense and sparse indices, respectively, for the *instructor* file. Suppose that we are looking up the record of instructor with *ID* “22222”. Using the dense index of Figure 14.2, we follow the pointer directly to the desired record. Since *ID* is a primary key, there exists only one such record and the search is complete. If we are using the sparse index (Figure 14.3), we do not find an index entry for “22222”. Since the last entry (in numerical order) before “22222” is “10101”, we follow that pointer. We then read the *instructor* file in sequential order until we find the desired record.

Consider a (printed) dictionary. The header of each page lists the first word alphabetically on that page. The words at the top of each page of the book index together form a sparse index on the contents of the dictionary pages.

As another example, suppose that the search-key value is not a primary key. Figure 14.4 shows a dense clustering index for the *instructor* file with the search key being *dept_name*. Observe that in this case the *instructor* file is sorted on the search key *dept_name*, instead of *ID*, otherwise the index on *dept_name* would be a nonclustering index. Suppose that we are looking up records for the History department. Using the dense index of Figure 14.4, we follow the pointer directly to the first History record. We process this record and follow the pointer in that record to locate the next record in

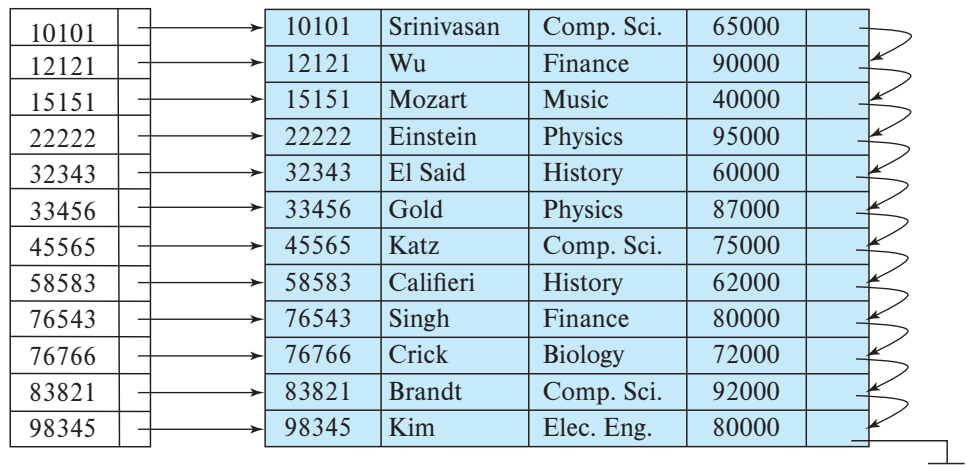


Figure 14.2 Dense index.

search-key (*dept_name*) order. We continue processing records until we encounter a record for a department other than History.

As we have seen, it is generally faster to locate a record if we have a dense index rather than a sparse index. However, sparse indices have advantages over dense indices in that they require less space and they impose less maintenance overhead for insertions and deletions.

There is a trade-off that the system designer must make between access time and space overhead. Although the decision regarding this trade-off depends on the specific application, a good compromise is to have a sparse index with one index entry per

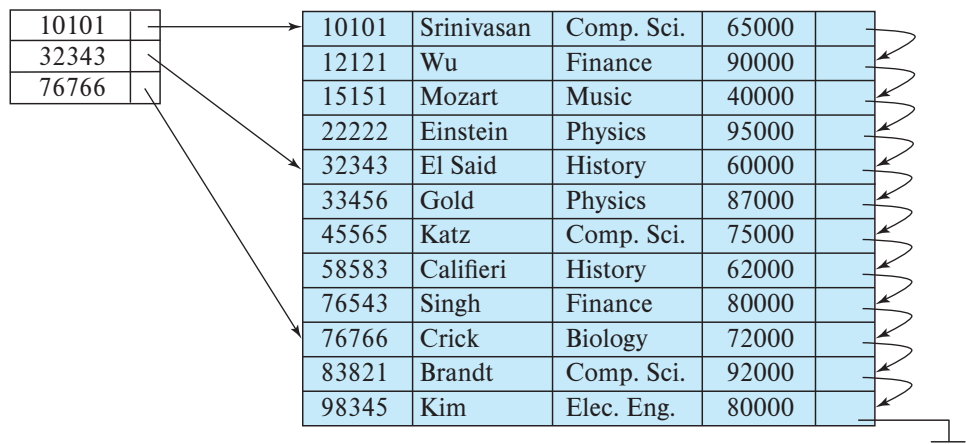


Figure 14.3 Sparse index.

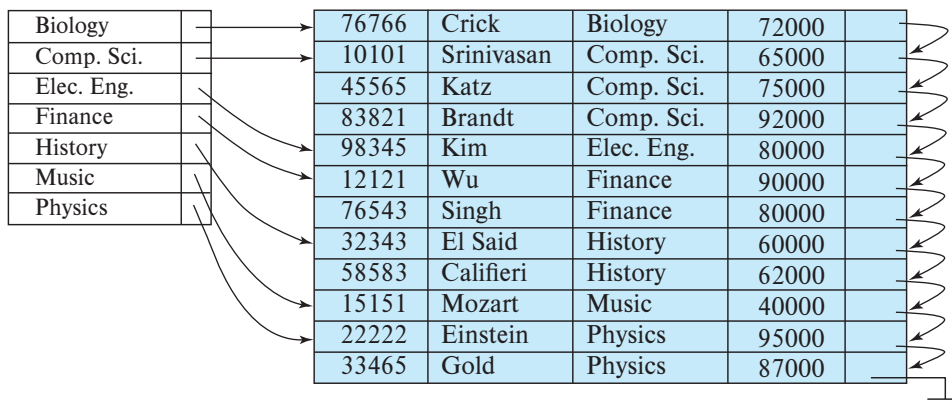


Figure 14.4 Dense index with search key *dept_name*.

block. The reason this design is a good trade-off is that the dominant cost in processing a database request is the time that it takes to bring a block from disk into main memory. Once we have brought in the block, the time to scan the entire block is negligible. Using this sparse index, we locate the block containing the record that we are seeking. Thus, unless the record is on an overflow block (see Section 13.3.2), we minimize block accesses while keeping the size of the index (and thus our space overhead) as small as possible.

For the preceding technique to be fully general, we must consider the case where records for one search-key value occupy several blocks. It is easy to modify our scheme to handle this situation.

14.2.2 Multilevel Indices

Suppose we build a dense index on a relation with 1,000,000 tuples. Index entries are smaller than data records, so let us assume that 100 index entries fit on a 4-kilobyte block. Thus, our index occupies 10,000 blocks. If the relation instead had 100,000,000 tuples, the index would instead occupy 1,000,000 blocks, or 4 gigabytes of space. Such large indices are stored as sequential files on disk.

If an index is small enough to be kept entirely in main memory, the search time to find an entry is low. However, if the index is so large that not all of it can be kept in memory, index blocks must be fetched from disk when required. (Even if an index is smaller than the main memory of a computer, main memory is also required for a number of other tasks, so it may not be possible to keep the entire index in memory.) The search for an entry in the index then requires several disk-block reads.

Binary search can be used on the index file to locate an entry, but the search still has a large cost. If the index would occupy b blocks, binary search requires as many as $\lceil \log_2(b) \rceil$ blocks to be read. ($\lceil x \rceil$ denotes the least integer that is greater than or equal to x ; that is, we round upward.) Note that the blocks that are read are not adjacent

to each other, so each read requires a random (i.e., non-sequential) I/O operation. For a 10,000-block index, binary search requires 14 random block reads. On a magnetic disk system where a random block read takes on average 10 milliseconds, the index search will take 140 milliseconds. This may not seem much, but we would be able to carry out only seven index searches a second on a single disk, whereas a more efficient search mechanism would let us carry out far more searches per second, as we shall see shortly. Note that, if overflow blocks have been used, binary search is only possible on the non-overflow blocks, and the actual cost may be even higher than the logarithmic bound above. A sequential search requires b sequential block reads, which may take even longer (although in some cases the lower cost of sequential block reads may result in sequential search being faster than a binary search). Thus, the process of searching a large index may be costly.

To deal with this problem, we treat the index just as we would treat any other sequential file, and we construct a sparse outer index on the original index, which we now call the inner index, as shown in Figure 14.5. Note that the index entries are always in sorted order, allowing the outer index to be sparse. To locate a record, we first use binary search on the outer index to find the record for the largest search-key value less

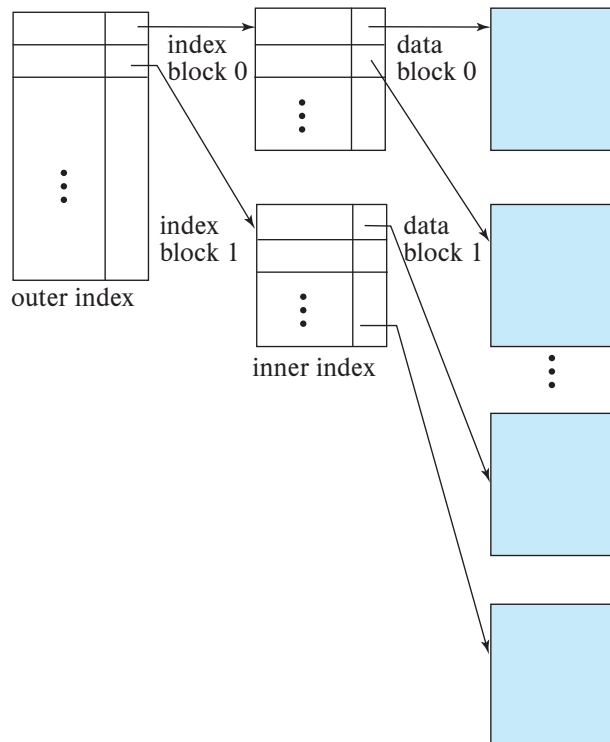


Figure 14.5 Two-level sparse index.

than or equal to the one that we desire. The pointer points to a block of the inner index. We scan this block until we find the record that has the largest search-key value less than or equal to the one that we desire. The pointer in this record points to the block of the file that contains the record for which we are looking.

In our example, an inner index with 10,000 blocks would require 10,000 entries in the outer index, which would occupy just 100 blocks. If we assume that the outer index is already in main memory, we would read only one index block for a search using a multilevel index, rather than the 14 blocks we read with binary search. As a result, we can perform 14 times as many index searches per second.

If our file is extremely large, even the outer index may grow too large to fit in main memory. With a 100,000,000-tuple relation, the inner index would occupy 1,000,000 blocks, and the outer index would occupy 10,000 blocks, or 40 megabytes. Since there are many demands on main memory, it may not be possible to reserve that much main memory just for this particular outer index. In such a case, we can create yet another level of index. Indeed, we can repeat this process as many times as necessary. Indices with two or more levels are called **multilevel indices**. Searching for records with a multilevel index requires significantly fewer I/O operations than does searching for records by binary search.²

Multilevel indices are closely related to tree structures, such as the binary trees used for in-memory indexing. We shall examine the relationship later, in Section 14.3.

14.2.3 Index Update

Regardless of what form of index is used, every index must be updated whenever a record is either inserted into or deleted from the file. Further, in case a record in the file is updated, any index whose search-key attribute is affected by the update must also be updated; for example, if the department of an instructor is changed, an index on the *dept_name* attribute of *instructor* must be updated correspondingly. Such a record update can be modeled as a deletion of the old record, followed by an insertion of the new value of the record, which results in an index deletion followed by an index insertion. As a result we only need to consider insertion and deletion on an index, and we do not need to consider updates explicitly.

We first describe algorithms for updating single-level indices.

14.2.3.1 Insertion

First, the system performs a lookup using the search-key value that appears in the record to be inserted. The actions the system takes next depend on whether the index is dense or sparse:

²In the early days of disk-based indices, each level of the index corresponded to a unit of physical storage. Thus, we may have indices at the track, cylinder, and disk levels. Such a hierarchy does not make sense today since disk subsystems hide the physical details of disk storage, and the number of disks and platters per disk is very small compared to the number of cylinders or bytes per track.

- Dense indices:
 1. If the search-key value does not appear in the index, the system inserts an index entry with the search-key value in the index at the appropriate position.
 2. Otherwise the following actions are taken:
 - a. If the index entry stores pointers to all records with the same search-key value, the system adds a pointer to the new record in the index entry.
 - b. Otherwise, the index entry stores a pointer to only the first record with the search-key value. The system then places the record being inserted after the other records with the same search-key values.
- Sparse indices: We assume that the index stores an entry for each block. If the system creates a new block, it inserts the first search-key value (in search-key order) appearing in the new block into the index. On the other hand, if the new record has the least search-key value in its block, the system updates the index entry pointing to the block; if not, the system makes no change to the index.

14.2.3.2 Deletion

To delete a record, the system first looks up the record to be deleted. The actions the system takes next depend on whether the index is dense or sparse:

- Dense indices:
 1. If the deleted record was the only record with its particular search-key value, then the system deletes the corresponding index entry from the index.
 2. Otherwise the following actions are taken:
 - a. If the index entry stores pointers to all records with the same search-key value, the system deletes the pointer to the deleted record from the index entry.
 - b. Otherwise, the index entry stores a pointer to only the first record with the search-key value. In this case, if the deleted record was the first record with the search-key value, the system updates the index entry to point to the next record.
- Sparse indices:
 1. If the index does not contain an index entry with the search-key value of the deleted record, nothing needs to be done to the index.
 2. Otherwise the system takes the following actions:
 - a. If the deleted record was the only record with its search key, the system replaces the corresponding index record with an index record for the next search-key value (in search-key order). If the next search-key value already has an index entry, the entry is deleted instead of being replaced.

- b. Otherwise, if the index entry for the search-key value points to the record being deleted, the system updates the index entry to point to the next record with the same search-key value.

Insertion and deletion algorithms for multilevel indices are a simple extension of the scheme just described. On deletion or insertion, the system updates the lowest-level index as described. As far as the second level is concerned, the lowest-level index is merely a file containing records—thus, if there is any change in the lowest-level index, the system updates the second-level index as described. The same technique applies to further levels of the index, if there are any.

14.2.4 Secondary Indices

Secondary indices must be dense, with an index entry for every search-key value, and a pointer to every record in the file. A clustering index may be sparse, storing only some of the search-key values, since it is always possible to find records with intermediate search-key values by a sequential access to a part of the file, as described earlier. If a secondary index stores only some of the search-key values, records with intermediate search-key values may be anywhere in the file and, in general, we cannot find them without searching the entire file.

A secondary index on a candidate key looks just like a dense clustering index, except that the records pointed to by successive values in the index are not stored sequentially. In general, however, secondary indices may have a different structure from clustering indices. If the search key of a clustering index is not a candidate key, it suffices if the index points to the first record with a particular value for the search key, since the other records can be fetched by a sequential scan of the file.

In contrast, if the search key of a secondary index is not a candidate key, it is not enough to point to just the first record with each search-key value. The remaining records with the same search-key value could be anywhere in the file, since the records are ordered by the search key of the clustering index, rather than by the search key of the secondary index. Therefore, a secondary index must contain pointers to all the records.

If a relation can have more than one record containing the same search key value (that is, two or more records can have the same values for the indexed attributes), the search key is said to be a **nonunique search key**.

One way to implement secondary indices on nonunique search keys is as follows: Unlike the case of primary indices, the pointers in such a secondary index do not point directly to the records. Instead, each pointer in the index points to a bucket that in turn contains pointers to the file. Figure 14.6 shows the structure of a secondary index that uses such an extra level of indirection on the *instructor* file, on the search key *dept_name*.

However, this approach has a few drawbacks. First, index access takes longer, due to an extra level of indirection, which may require a random I/O operation. Second,

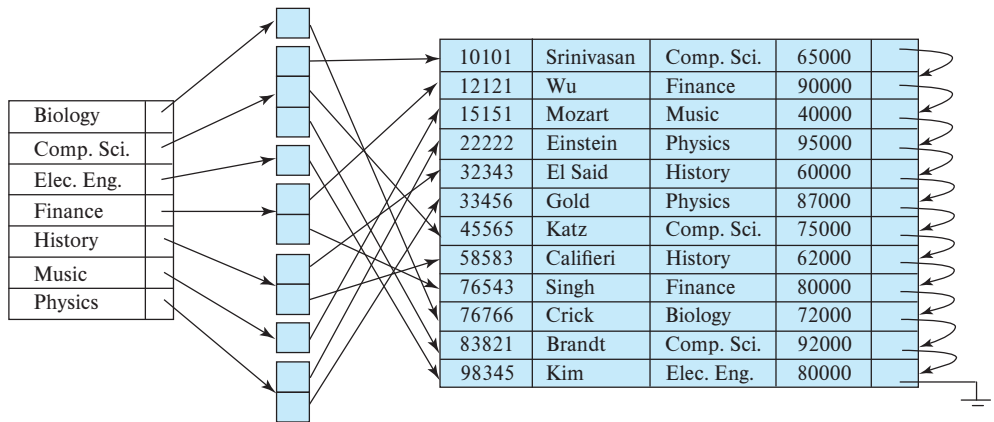


Figure 14.6 Secondary index on *instructor* file, on noncandidate key *dept_name*.

if a key has very few or no duplicates, if a whole block is allocated to its associated bucket, a lot of space would be wasted. Later in this chapter, we study more efficient alternatives for implementing secondary indices, which avoid these drawbacks.

A sequential scan in clustering index order is efficient because records in the file are stored physically in the same order as the index order. However, we cannot (except in rare special cases) store a file physically ordered by both the search key of the clustering index and the search key of a secondary index. Because secondary-key order and physical-key order differ, if we attempt to scan the file sequentially in secondary-key order, the reading of each record is likely to require the reading of a new block from disk, which is very slow.

The procedure described earlier for deletion and insertion can also be applied to secondary indices; the actions taken are those described for dense indices storing a pointer to every record in the file. If a file has multiple indices, whenever the file is modified, *every* index must be updated.

Secondary indices improve the performance of queries that use keys other than the search key of the clustering index. However, they impose a significant overhead on modification of the database. The designer of a database decides which secondary indices are desirable on the basis of an estimate of the relative frequency of queries and modifications.

14.2.5 Indices on Multiple Keys

Although the examples we have seen so far have had a single attribute in a search key, in general a search key can have more than one attribute. A search key containing more than one attribute is referred to as a **composite search key**. The structure of the index is the same as that of any other index, the only difference being that the search key is not a single attribute, but rather is a list of attributes. The search key can be represented as a tuple of values, of the form $(a_1, \dots, a_n)$, where the indexed attributes are $A_1, \dots, A_n$.

The ordering of search-key values is the *lexicographic ordering*. For example, for the case of two attribute search keys, $(a_1, a_2) < (b_1, b_2)$ if either $a_1 < b_1$ or $a_1 = b_1$ and $a_2 < b_2$. Lexicographic ordering is basically the same as alphabetic ordering of words.

As an example, consider an index on the *takes* relation, on the composite search key $(course_id, semester, year)$. Such an index would be useful to find all students who have registered for a particular course in a particular semester/year. An ordered index on a composite key can also be used to answer several other kinds of queries efficiently, as we shall see in Section 14.6.2.

14.3 B⁺-Tree Index Files

The main disadvantage of the index-sequential file organization is that performance degrades as the file grows, both for index lookups and for sequential scans through the data. Although this degradation can be remedied by reorganization of the file, frequent reorganizations are undesirable.

The **B⁺-tree index** structure is the most widely used of several index structures that maintain their efficiency despite insertion and deletion of data. A B⁺-tree index takes the form of a **balanced tree** in which every path from the root of the tree to a leaf of the tree is of the same length. Each nonleaf node in the tree (other than the root) has between $\lceil n/2 \rceil$ and n children, where n is fixed for a particular tree; the root has between 2 and n children.

We shall see that the B⁺-tree structure imposes performance overhead on insertion and deletion and adds space overhead. The overhead is acceptable even for frequently modified files, since the cost of file reorganization is avoided. Furthermore, since nodes may be as much as half empty (if they have the minimum number of children), there is some wasted space. This space overhead, too, is acceptable given the performance benefits of the B⁺-tree structure.

14.3.1 Structure of a B⁺-Tree

A B⁺-tree index is a multilevel index, but it has a structure that differs from that of the multilevel index-sequential file. We assume for now that there are no duplicate search key values, that is, each search key is unique and occurs in at most one record; we consider the issue of nonunique search keys later.

Figure 14.7 shows a typical node of a B⁺-tree. It contains up to $n - 1$ search-key values $K_1, K_2, \dots, K_{n-1}$, and n pointers $P_1, P_2, \dots, P_n$. The search-key values within a node are kept in sorted order; thus, if $i < j$, then $K_i < K_j$.



Figure 14.7 Typical node of a B⁺-tree.

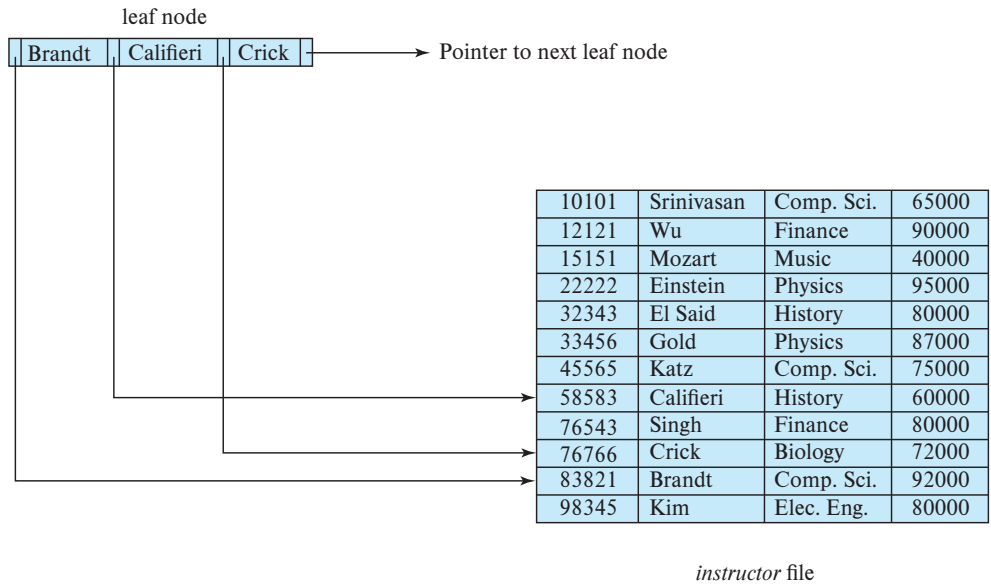


Figure 14.8 A leaf node for *instructor* B⁺-tree index ($n = 4$).

We consider first the structure of the **leaf nodes**. For $i = 1, 2, \dots, n - 1$, pointer P_i points to a file record with search-key value K_i . Pointer P_n has a special purpose that we shall discuss shortly.

Figure 14.8 shows one leaf node of a B⁺-tree for the *instructor* file, in which we have chosen n to be 4, and the search key is *name*.

Now that we have seen the structure of a leaf node, let us consider how search-key values are assigned to particular nodes. Each leaf can hold up to $n - 1$ values. We allow leaf nodes to contain as few as $\lceil (n - 1)/2 \rceil$ values. With $n = 4$ in our example B⁺-tree, each leaf must contain at least two values, and at most three values.

If L_i and L_j are leaf nodes and $i < j$ (that is, L_i is to the left of L_j in the tree), then every search-key value v_i in L_i is less than every search-key value v_j in L_j .

If the B⁺-tree index is used as a dense index (as is usually the case), every search-key value must appear in some leaf node.

Now we can explain the use of the pointer P_n . Since there is a linear order on the leaves based on the search-key values that they contain, we use P_n to chain together the leaf nodes in search-key order. This ordering allows for efficient sequential processing of the file.

The **nonleaf nodes** of the B⁺-tree form a multilevel (sparse) index on the leaf nodes. The structure of nonleaf nodes is the same as that for leaf nodes, except that all pointers are pointers to tree nodes. A nonleaf node may hold up to n pointers and *must* hold at least $\lceil n/2 \rceil$ pointers. The number of pointers in a node is called the *fanout* of the node. Nonleaf nodes are also referred to as **internal nodes**.

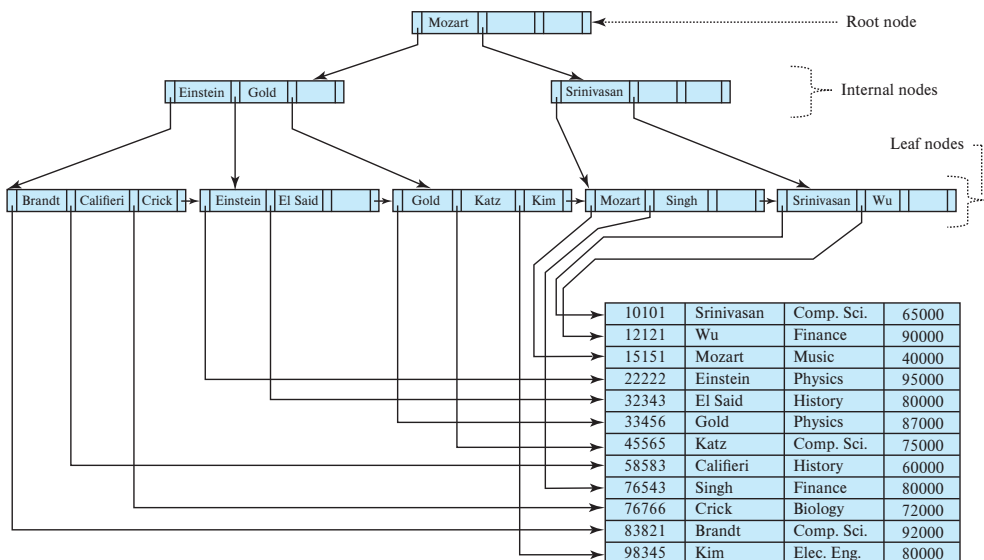


Figure 14.9 B⁺-tree for *instructor* file ($n = 4$).

Let us consider a node containing m pointers ($m \leq n$). For $i = 2, 3, \dots, m - 1$, pointer P_i points to the subtree that contains search-key values less than K_i and greater than or equal to K_{i-1} . Pointer P_m points to the part of the subtree that contains those key values greater than or equal to K_{m-1} , and pointer P_1 points to the part of the subtree that contains those search-key values less than K_1 .

Unlike other nonleaf nodes, the root node can hold fewer than $\lceil n/2 \rceil$ pointers; however, it must hold at least two pointers, unless the tree consists of only one node. It is always possible to construct a B⁺-tree, for any n , that satisfies the preceding requirements.

Figure 14.9 shows a complete B⁺-tree for the *instructor* file (with $n = 4$). We have omitted null pointers for simplicity; any pointer field in the figure that does not have an arrow is understood to have a null value.

Figure 14.10 shows another B⁺-tree for the *instructor* file, this time with $n = 6$. Observe that the height of this tree is less than that of the previous tree, which had $n = 4$.

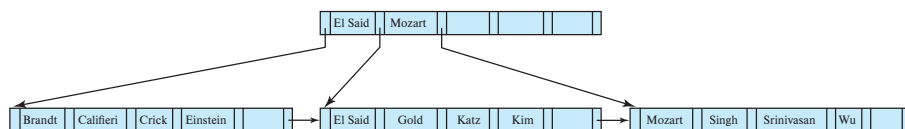


Figure 14.10 B⁺-tree for *instructor* file with $n = 6$.

These examples of B⁺-trees are all balanced. That is, the length of every path from the root to a leaf node is the same. This property is a requirement for a B⁺-tree. Indeed, the “B” in B⁺-tree stands for “balanced.” It is the balance property of B⁺-trees that ensures good performance for lookup, insertion, and deletion.

In general, search keys could have duplicates. One way to handle the case of nonunique search keys is to modify the tree structure to store each search key at a leaf node as many times as it appears in records, with each copy pointing to one record. The condition that $K_i < K_j$ if $i < j$ will need to be modified to $K_i \leq K_j$. However, this approach can result in duplicate search key values at internal nodes, making the insertion and deletion procedures more complicated and expensive. Another alternative is to store a set (or bucket) of record pointers with each search key value, as we saw earlier. This approach is more complicated and can result in inefficient access, especially if the number of record pointers for a particular key is very large.

Most database implementations instead make search keys unique as follows: Suppose the desired search key attribute a_i of relation r is nonunique. Let A_p be the primary key of r . Then the unique composite search key (a_i, A_p) is used instead of a_i when building the index. (Any set of attributes that together with a_i guarantee uniqueness can also be used instead of A_p .) For example, if we wished to create an index on the *instructor* relation on the attribute *name*, we instead create an index on the composite search key $(name, ID)$, since ID is the primary key for *instructor*. Index lookups on just *name* can be efficiently handled using this index, as we shall see shortly. Section 14.3.5 covers issues in handling of nonunique search keys in more detail.

In our examples, we show indices on some nonunique search keys, such as *instructor.name*, assuming for simplicity that there are no duplicates; in reality most databases would automatically add extra attributes internally, to ensure the absence of duplicates.

14.3.2 Queries on B⁺-Trees

Let us consider how we process queries on a B⁺-tree. Suppose that we wish to find a record with a given value v for the search key. Figure 14.11 presents pseudocode for a function *find*(v) to carry out this task, assuming there are no duplicates, that is, there is at most one record with a particular search key. We address the issue of nonunique search keys later in this section.

Intuitively, the function starts at the root of the tree and traverses the tree down until it reaches a leaf node that would contain the specified value if it exists in the tree. Specifically, starting with the root as the current node, the function repeats the following steps until a leaf node is reached. First, the current node is examined, looking for the smallest i such that search-key value K_i is greater than or equal to v . Suppose such a value is found; then, if K_i is equal to v , the current node is set to the node pointed to by P_{i+1} , otherwise $K_i > v$, and the current node is set to the node pointed to by P_i . If no such value K_i is found, then $v > K_{m-1}$, where P_m is the last nonnull pointer in the node. In this case the current node is set to that pointed to by P_m . The above procedure is repeated, traversing down the tree until a leaf node is reached.

```

function find(v)
/* Assumes no duplicate keys, and returns pointer to the record with
 * search key value v if such a record exists, and null otherwise */
  Set C = root node
  while (C is not a leaf node) begin
    Let i = smallest number such that  $v \leq C.K_i$ 
    if there is no such number i then begin
      Let  $P_m$  = last non-null pointer in the node
      Set  $C = C.P_m$ 
    end
    else if ( $v = C.K_i$ ) then Set  $C = C.P_{i+1}$ 
    else Set  $C = C.P_i$  /*  $v < C.K_i$  */
  end
/* C is a leaf node */
if for some i,  $K_i = v$ 
  then return  $P_i$ 
else return null ; /* No record with key value v exists */

```

Figure 14.11 Querying a B⁺-tree.

At the leaf node, if there is a search-key value $K_i = v$, pointer P_i directs us to a record with search-key value K_i . The function then returns the pointer to the record, P_i . If no search key with value v is found in the leaf node, no record with key value v exists in the relation, and function *find* returns null, to indicate failure.

B⁺-trees can also be used to find all records with search key values in a specified range $[lb, ub]$. For example, with a B⁺-tree on attribute *salary* of *instructor*, we can find all *instructor* records with salary in a specified range such as [50000, 100000] (in other words, all salaries between 50000 and 100000). Such queries are called **range queries**.

To execute such queries, we can create a procedure *findRange* (*lb*, *ub*), shown in Figure 14.12. The procedure does the following: it first traverses to a leaf in a manner similar to *find*(*lb*); the leaf may or may not actually contain value *lb*. It then steps through records in that and subsequent leaf nodes collecting pointers to all records with key values $C.K_i$ s.t. $lb \leq C.K_i \leq ub$ into a set *resultSet*. The function stops when $C.K_i > ub$, or there are no more keys in the tree.

A real implementation would provide a version of *findRange* supporting an iterator interface similar to that provided by the JDBC *ResultSet*, which we saw in Section 5.1.1. Such an iterator interface would provide a method *next*(), which can be called repeatedly to fetch successive records. The *next*() method would step through the entries at the leaf level, in a manner similar to *findRange*, but each call takes only one step and records where it left off, so that successive calls to *next*() step through successive en-

```

function findRange(lb, ub)
/* Returns all records with search key value V such that  $lb \leq V \leq ub$ . */
  Set resultSet = {};
  Set C = root node
  while (C is not a leaf node) begin
    Let i = smallest number such that  $lb \leq C.K_i$ 
    if there is no such number i then begin
      Let  $P_m$  = last non-null pointer in the node
      Set  $C = C.P_m$ 
    end
    else if ( $lb = C.K_i$ ) then Set  $C = C.P_{i+1}$ 
    else Set  $C = C.P_i$  /*  $lb < C.K_i$  */
  end
  /* C is a leaf node */
  Let i be the least value such that  $K_i \geq lb$ 
  if there is no such i
    then Set i = 1 + number of keys in C; /* To force move to next leaf */
  Set done = false;
  while (not done) begin
    Let n = number of keys in C.
    if ( $i \leq n$  and  $C.K_i \leq ub$ ) then begin
      Add  $C.P_i$  to resultSet
      Set  $i = i + 1$ 
    end
    else if ( $i \leq n$  and  $C.K_i > ub$ )
      then Set done = true;
    else if ( $i > n$  and  $C.P_{n+1}$  is not null)
      then Set  $C = C.P_{n+1}$ , and  $i = 1$  /* Move to next leaf */
    else Set done = true; /* No more leaves to the right */
  end
return resultSet;

```

Figure 14.12 Range query on a B⁺-tree.

tries. We omit details for simplicity, and leave the pseudocode for the iterator interface as an exercise for the interested reader.

We now consider the cost of querying on a B⁺-tree index. In processing a query, we traverse a path in the tree from the root to some leaf node. If there are N records in the file, the path is no longer than $\lceil \log_{\lceil n/2 \rceil}(N) \rceil$.

Typically, the node size is chosen to be the same as the size of a disk block, which is typically 4 kilobytes. With a search-key size of 12 bytes, and a disk-pointer size of

8 bytes, n is around 200. Even with a more conservative estimate of 32 bytes for the search-key size, n is around 100. With $n = 100$, if we have 1 million search-key values in the file, a lookup requires only $\lceil \log_{50}(1,000,000) \rceil = 4$ nodes to be accessed. Thus, at most four blocks need to be read from disk to traverse the path from the root to a leaf. The root node of the tree is usually heavily accessed and is likely to be in the buffer, so typically only three or fewer blocks need to be read from disk.

An important difference between B⁺-tree structures and in-memory tree structures, such as binary trees, is the size of a node, and as a result, the height of the tree. In a binary tree, each node is small and has at most two pointers. In a B⁺-tree, each node is large—typically a disk block—and a node can have a large number of pointers. Thus, B⁺-trees tend to be fat and short, unlike thin and tall binary trees. In a balanced binary tree, the path for a lookup can be of length $\lceil \log_2(N) \rceil$, where N is the number of records in the file being indexed. With $N = 1,000,000$ as in the previous example, a balanced binary tree requires around 20 node accesses. If each node were on a different disk block, 20 block reads would be required to process a lookup, in contrast to the four block reads for the B⁺-tree. The difference is significant with a magnetic disk, since each block read could require a disk arm seek which, together with the block read, takes about 10 milliseconds on a magnetic disk. The difference is not quite as drastic with flash storage, where a read of a 4 kilobyte page takes around 10 to 100 microseconds, but it is still significant.

After traversing down to the leaf level, queries on a single value of a unique search key require one more random I/O operation to fetch any matching record.

Range queries have an additional cost, after traversing down to the leaf level: all the pointers in the given range must be retrieved. These pointers are in consecutive leaf nodes; thus, if M such pointers are retrieved, at most $\lceil M/(n/2) \rceil + 1$ leaf nodes need to be accessed to retrieve the pointers (since each leaf node has at least $n/2$ pointers, but even two pointers may be split across two pages). To this cost, we need to add the cost of accessing the actual records. For secondary indices, each such record may be on a different block, which could result in M random I/O operations in the worst case. For clustered indices, these records would be in consecutive blocks, with each block containing multiple records, resulting in a significantly lower cost.

Now, let us consider the case of nonunique keys. As explained earlier, if we wish to create an index on an attribute a_i that is not a candidate key, and may thus have duplicates, we instead create an index on a composite key that is duplicate-free. The composite key is created by adding extra attributes, such as the primary key, to a_i , to ensure uniqueness. Suppose we created an index on the composite key (a_i, A_p) instead of creating an index on a_i .

An important question, then, is how do we retrieve all tuples with a given value v for a_i using the above index? This question is easily answered by using the function $\text{findRange}(lb, ub)$, with $lb = (v, -\infty)$ and $ub = (v, \infty)$, where $-\infty$ and ∞ denote the smallest and largest possible values of A_p . All records with $a_i = v$ would be returned by the above function call. Range queries on a_i can be handled similarly. These range

queries retrieve pointers to the records quite efficiently, although retrieval of the records may be expensive, as discussed earlier.

14.3.3 Updates on B⁺-Trees

When a record is inserted into, or deleted from a relation, indices on the relation must be updated correspondingly. Recall that updates to a record can be modeled as a deletion of the old record followed by insertion of the updated record. Hence we only consider the case of insertion and deletion.

Insertion and deletion are more complicated than lookup, since it may be necessary to **split** a node that becomes too large as the result of an insertion, or to **coalesce** nodes (i.e., combine nodes) if a node becomes too small (fewer than $\lceil n/2 \rceil$ pointers). Furthermore, when a node is split or a pair of nodes is combined, we must ensure that balance is preserved. To introduce the idea behind insertion and deletion in a B⁺-tree, we shall assume temporarily that nodes never become too large or too small. Under this assumption, insertion and deletion are performed as defined next.

- **Insertion.** Using the same technique as for lookup from the *find()* function (Figure 14.11), we first find the leaf node in which the search-key value would appear. We then insert an entry (i.e., a search-key value and record pointer pair) in the leaf node, positioning it such that the search keys are still in order.
- **Deletion.** Using the same technique as for lookup, we find the leaf node containing the entry to be deleted by performing a lookup on the search-key value of the deleted record; if there are multiple entries with the same search-key value, we search across all entries with the same search-key value until we find the entry that points to the record being deleted. We then remove the entry from the leaf node. All entries in the leaf node that are to the right of the deleted entry are shifted left by one position, so that there are no gaps in the entries after the entry is deleted.

We now consider the general case of insertion and deletion, dealing with node splitting and node coalescing.

14.3.3.1 Insertion

We now consider an example of insertion in which a node must be split. Assume that a record is inserted on the *instructor* relation, with the *name* value being Adams. We then need to insert an entry for “Adams” into the B⁺-tree of Figure 14.9. Using the algorithm for lookup, we find that “Adams” should appear in the leaf node containing “Brandt”, “Califieri”, and “Crick.” There is no room in this leaf to insert the search-key value “Adams.” Therefore, the node is *split* into two nodes. Figure 14.13 shows the two leaf nodes that result from the split of the leaf node on inserting “Adams”. The search-key values “Adams” and “Brandt” are in one leaf, and “Califieri” and “Crick” are in the other. In general, we take the n search-key values (the $n - 1$ values in the leaf

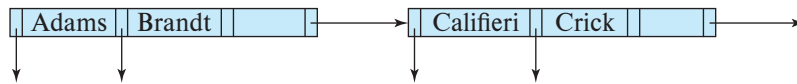


Figure 14.13 Split of leaf node on insertion of “Adams”.

node plus the value being inserted), and put the first $\lceil n/2 \rceil$ in the existing node and the remaining values in a newly created node.

Having split a leaf node, we must insert the new leaf node into the B⁺-tree structure. In our example, the new node has “Califieri” as its smallest search-key value. We need to insert an entry with this search-key value, and a pointer to the new node, into the parent of the leaf node that was split. The B⁺-tree of Figure 14.14 shows the result of the insertion. It was possible to perform this insertion with no further node split, because there was room in the parent node for the new entry. If there were no room, the parent would have had to be split, requiring an entry to be added to its parent. In the worst case, all nodes along the path to the root must be split. If the root itself is split, the entire tree becomes deeper.

Splitting of a nonleaf node is a little different from splitting of a leaf node. Figure 14.15 shows the result of inserting a record with search key “Lamport” into the tree shown in Figure 14.14. The leaf node in which “Lamport” is to be inserted already has entries “Gold”, “Katz”, and “Kim”, and as a result the leaf node has to be split. The new right-hand-side node resulting from the split contains the search-key values “Kim” and “Lamport”. An entry (Kim, $n1$) must then be added to the parent node, where $n1$ is a pointer to the new node. However, there is no space in the parent node to add a new entry, and the parent node has to be split. To do so, the parent node is conceptually expanded temporarily, the entry added, and the overfull node is then immediately split.

When an overfull nonleaf node is split, the child pointers are divided among the original and the newly created nodes; in our example, the original node is left with the first three pointers, and the newly created node to the right gets the remaining two pointers. The search key values are, however, handled a little differently. The search key values that lie between the pointers moved to the right node (in our example, the value “Kim”) are moved along with the pointers, while those that lie between the pointers that stay on the left (in our example, “Califieri” and “Einstein”) remain undisturbed.

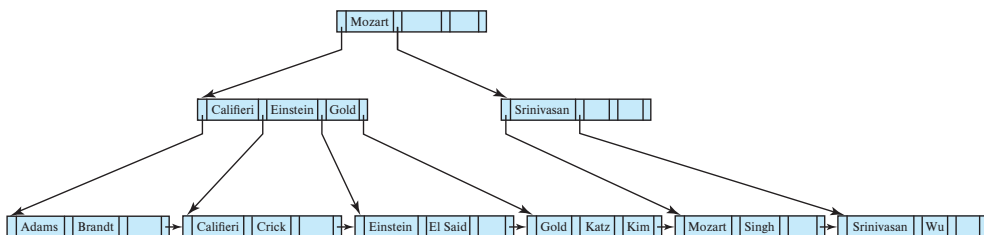


Figure 14.14 Insertion of “Adams” into the B⁺-tree of Figure 14.9.

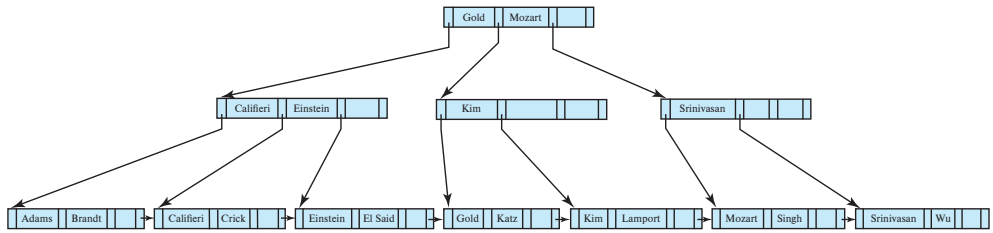


Figure 14.15 Insertion of “Lampport” into the B⁺-tree of Figure 14.14.

However, the search key value that lies between the pointers that stay on the left, and the pointers that move to the right node is treated differently. In our example, the search key value “Gold” lies between the three pointers that went to the left node, and the two pointers that went to the right node. The value “Gold” is not added to either of the split nodes. Instead, an entry (Gold, n_2) is added to the parent node, where n_2 is a pointer to the newly created node that resulted from the split. In this case, the parent node is the root, and it has enough space for the new entry.

The general technique for insertion into a B⁺-tree is to determine the leaf node l into which insertion must occur. If a split results, insert the new node into the parent

```

procedure insert(value  $K$ , pointer  $P$ )
  if (tree is empty) create an empty leaf node  $L$ , which is also the root
  else Find the leaf node  $L$  that should contain key value  $K$ 
  if ( $L$  has less than  $n - 1$  key values)
    then insert_in_leaf ( $L$ ,  $K$ ,  $P$ )
  else begin /*  $L$  has  $n - 1$  key values already, split it */
    Create node  $L'$ 
    Copy  $L.P_1 \dots L.K_{n-1}$  to a block of memory  $T$  that can
      hold  $n$  (pointer, key-value) pairs
    insert_in_leaf ( $T$ ,  $K$ ,  $P$ )
    Set  $L'.P_n = L.P_n$ ; Set  $L.P_n = L'$ 
    Erase  $L.P_1$  through  $L.K_{n-1}$  from  $L$ 
    Copy  $T.P_1$  through  $T.K_{\lceil n/2 \rceil}$  from  $T$  into  $L$  starting at  $L.P_1$ 
    Copy  $T.P_{\lceil n/2 \rceil + 1}$  through  $T.K_n$  from  $T$  into  $L'$  starting at  $L'.P_1$ 
    Let  $K'$  be the smallest key-value in  $L'$ 
    insert_in_parent( $L$ ,  $K'$ ,  $L'$ )
  end

```

Figure 14.16 Insertion of entry in a B⁺-tree.

of node l . If this insertion causes a split, proceed recursively up the tree until either an insertion does not cause a split or a new root is created.

Figure 14.16 outlines the insertion algorithm in pseudocode. The procedure *insert* inserts a key-value pointer pair into the index, using two subsidiary procedures *insert_in_Leaf* and *insert_in_parent*, shown in Figure 14.17. In the pseudocode, L, N, P and T denote pointers to nodes, with L being used to denote a leaf node. $L.K_i$ and $L.P_i$ denote the i th value and the i th pointer in node L , respectively; $T.K_i$ and $T.P_i$ are used similarly. The pseudocode also makes use of the function *parent*(N) to find the parent of a node N . We can compute a list of nodes in the path from the root to the leaf while initially finding the leaf node, and we can use it later to find the parent of any node in the path efficiently.

```

procedure insert_in_Leaf (node  $L$ , value  $K$ , pointer  $P$ )
    if ( $K < L.K_1$ )
        then insert  $P, K$  into  $L$  just before  $L.P_1$ 
    else begin
        Let  $K_i$  be the highest value in  $L$  that is less than or equal to  $K$ 
        Insert  $P, K$  into  $L$  just after  $L.K_i$ 
    end

procedure insert_in_parent(node  $N$ , value  $K'$ , node  $N'$ )
    if ( $N$  is the root of the tree)
        then begin
            Create a new node  $R$  containing  $N, K', N'$  /*  $N$  and  $N'$  are pointers */
            Make  $R$  the root of the tree
            return
        end
    Let  $P = \text{parent}(N)$ 
    if ( $P$  has less than  $n$  pointers)
        then insert ( $K', N'$ ) in  $P$  just after  $N$ 
    else begin /* Split  $P$  */
        Copy  $P$  to a block of memory  $T$  that can hold  $P$  and ( $K', N'$ )
        Insert ( $K', N'$ ) into  $T$  just after  $N$ 
        Erase all entries from  $P$ ; Create node  $P'$ 
        Copy  $T.P_1 \dots T.P_{\lceil (n+1)/2 \rceil}$  into  $P$ 
        Let  $K'' = T.K_{\lceil (n+1)/2 \rceil}$ 
        Copy  $T.P_{\lceil (n+1)/2 \rceil + 1} \dots T.P_{n+1}$  into  $P'$ 
        insert_in_parent( $P, K'', P'$ )
    end

```

Figure 14.17 Subsidiary procedures for insertion of entry in a B^+ -tree.

The procedure *insert_in_parent* takes as parameters N, K', N' , where node N was split into N and N' , with K' being the least value in N' . The procedure modifies the parent of N to record the split. The procedures *insert_into_index* and *insert_in_parent* use a temporary area of memory T to store the contents of a node being split. The procedures can be modified to copy data from the node being split directly to the newly created node, reducing the time required for copying data. However, the use of the temporary space T simplifies the procedures.

14.3.3.2 Deletion

We now consider deletions that cause tree nodes to contain too few pointers. First, let us delete “Srinivasan” from the B⁺-tree of Figure 14.14. The resulting B⁺-tree appears in Figure 14.18. We now consider how the deletion is performed. We first locate the entry for “Srinivasan” by using our lookup algorithm. When we delete the entry for “Srinivasan” from its leaf node, the node is left with only one entry, “Wu”. Since, in our example, $n = 4$ and $1 < \lceil (n - 1)/2 \rceil$, we must either merge the node with a sibling node or redistribute the entries between the nodes, to ensure that each node is at least half-full. In our example, the underfull node with the entry for “Wu” can be merged with its left sibling node. We merge the nodes by moving the entries from both the nodes into the left sibling and deleting the now-empty right sibling. Once the node is deleted, we must also delete the entry in the parent node that pointed to the just deleted node.

In our example, the entry to be deleted is (Srinivasan, n_3), where n_3 is a pointer to the leaf containing “Srinivasan”. (In this case the entry to be deleted in the nonleaf node happens to be the same value as that deleted from the leaf; that would not be the case for most deletions.) After deleting the above entry, the parent node, which had a search key value “Srinivasan” and two pointers, now has one pointer (the leftmost pointer in the node) and no search-key values. Since $1 < \lceil n/2 \rceil$ for $n = 4$, the parent node is underfull. (For larger n , a node that becomes underfull would still have some values as well as pointers.)

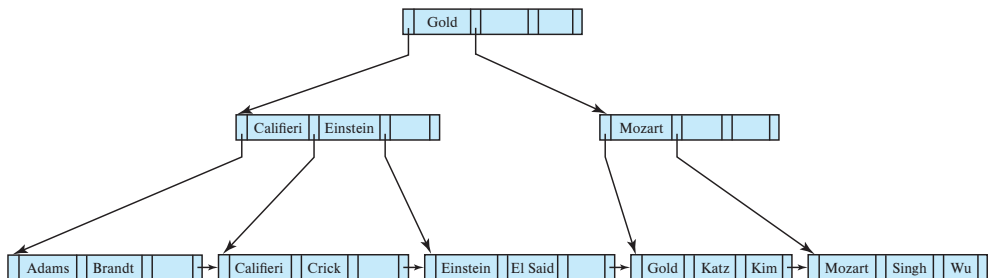


Figure 14.18 Deletion of “Srinivasan” from the B⁺-tree of Figure 14.14.

In this case, we look at a sibling node; in our example, the only sibling is the nonleaf node containing the search keys “Califieri”, “Einstein”, and “Gold”. If possible, we try to coalesce the node with its sibling. In this case, coalescing is not possible, since the node and its sibling together have five pointers, against a maximum of four. The solution in this case is to **redistribute** the pointers between the node and its sibling, such that each has at least $\lceil n/2 \rceil = 2$ child pointers. To do so, we move the rightmost pointer from the left sibling (the one pointing to the leaf node containing “Gold”) to the underfull right sibling. However, the underfull right sibling would now have two pointers, namely, its leftmost pointer, and the newly moved pointer, with no value separating them. In fact, the value separating them is not present in either of the nodes, but is present in the parent node, between the pointers from the parent to the node and its sibling. In our example, the value “Mozart” separates the two pointers and is present in the right sibling after the redistribution. Redistribution of the pointers also means that the value “Mozart” in the parent no longer correctly separates search-key values in the two siblings. In fact, the value that now correctly separates search-key values in the two sibling nodes is the value “Gold”, which was in the left sibling before redistribution.

As a result, as can be seen in the B⁺-tree in Figure 14.18, after redistribution of pointers between siblings, the value “Gold” has moved up into the parent, while the value that was there earlier, “Mozart”, has moved down into the right sibling.

We next delete the search-key values “Singh” and “Wu” from the B⁺-tree of Figure 14.18. The result is shown in Figure 14.19. The deletion of the first of these values does not make the leaf node underfull, but the deletion of the second value does. It is not possible to merge the underfull node with its sibling, so a redistribution of values is carried out, moving the search-key value “Kim” into the node containing “Mozart”, resulting in the tree shown in Figure 14.19. The value separating the two siblings has been updated in the parent, from “Mozart” to “Kim”.

Now we delete “Gold” from the above tree; the result is shown in Figure 14.20. This results in an underfull leaf, which can now be merged with its sibling. The resultant deletion of an entry from the parent node (the nonleaf node containing “Kim”) makes the parent underfull (it is left with just one pointer). This time around, the parent node can be merged with its sibling. This merge results in the search-key value “Gold”

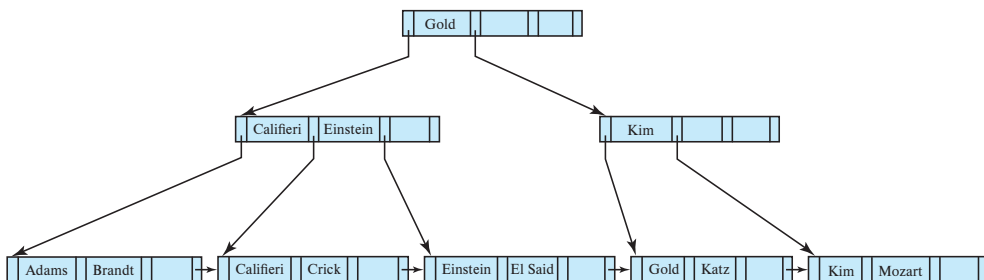


Figure 14.19 Deletion of “Singh” and “Wu” from the B⁺-tree of Figure 14.18.

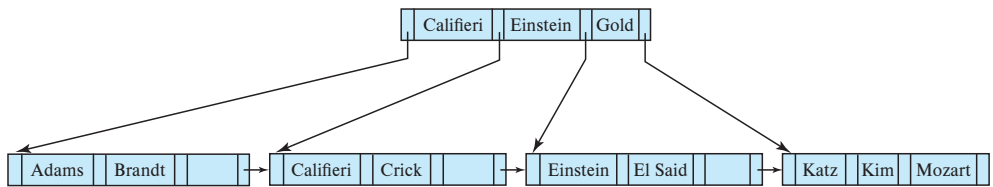


Figure 14.20 Deletion of “Gold” from the B⁺-tree of Figure 14.19.

moving down from the parent into the merged node. As a result of this merge, an entry is deleted from its parent, which happens to be the root of the tree. And as a result of that deletion, the root is left with only one child pointer and no search-key value, violating the condition that the root must have at least two children. As a result, the root node is deleted and its sole child becomes the root, and the depth of the B⁺-tree has been decreased by 1.

It is worth noting that, as a result of deletion, a key value that is present in a nonleaf node of the B⁺-tree may not be present at any leaf of the tree. For example, in Figure 14.20, the value “Gold” has been deleted from the leaf level but is still present in a nonleaf node.

In general, to delete a value in a B⁺-tree, we perform a lookup on the value and delete it. If the node is too small, we delete it from its parent. This deletion results in recursive application of the deletion algorithm until the root is reached, a parent remains adequately full after deletion, or redistribution is applied.

Figure 14.21 outlines the pseudocode for deletion from a B⁺-tree. The procedure *swap_variables*(*N*, *N'*) merely swaps the values of the (pointer) variables *N* and *N'*; this swap has no effect on the tree itself. The pseudocode uses the condition “too few pointers/values.” For nonleaf nodes, this criterion means less than $\lceil n/2 \rceil$ pointers; for leaf nodes, it means less than $\lceil (n-1)/2 \rceil$ values. The pseudocode redistributes entries by borrowing a single entry from an adjacent node. We can also redistribute entries by repartitioning entries equally between the two nodes. The pseudocode refers to deleting an entry (*K*, *P*) from a node. In the case of leaf nodes, the pointer to an entry actually precedes the key value, so the pointer *P* precedes the key value *K*. For nonleaf nodes, *P* follows the key value *K*.

14.3.4 Complexity of B⁺-Tree Updates

Although insertion and deletion operations on B⁺-trees are complicated, they require relatively few I/O operations, which is an important benefit since I/O operations are expensive. It can be shown that the number of I/O operations needed in the worst case for an insertion is proportional to $\log_{\lceil n/2 \rceil}(N)$, where *n* is the maximum number of pointers in a node, and *N* is the number of records in the file being indexed.

The worst-case complexity of the deletion procedure is also proportional to $\log_{\lceil n/2 \rceil}(N)$, provided there are no duplicate values for the search key; we discuss the case of nonunique search keys later in this chapter.

```

procedure delete(value K, pointer P)
    find the leaf node L that contains (K, P)
    delete_entry(L, K, P)

procedure delete_entry(node N, value K, pointer P)
    delete (K, P) from N
    if (N is the root and N has only one remaining child)
    then make the child of N the new root of the tree and delete N
    else if (N has too few values/pointers) then begin
        Let N' be the previous or next child of parent(N)
        Let K' be the value between pointers N and N' in parent(N)
        if (entries in N and N' can fit in a single node)
            then begin /* Coalesce nodes */
                if (N is a predecessor of N') then swap_variables(N, N')
                if (N is not a leaf)
                    then append K' and all pointers and values in N to N'
                    else append all (Ki, Pi) pairs in N to N'; set N'.Pn = N.Pn
                delete_entry(parent(N), K', N); delete node N
            end
        else begin /* Redistribution: borrow an entry from N' */
            if (N' is a predecessor of N) then begin
                if (N is a nonleaf node) then begin
                    let m be such that N'.Pm is the last pointer in N'
                    remove (N'.Km-1, N'.Pm) from N'
                    insert (N'.Pm, K') as the first pointer and value in N,
                        by shifting other pointers and values right
                    replace K' in parent(N) by N'.Km-1
                end
            else begin
                let m be such that (N'.Pm, N'.Km) is the last pointer/value
                    pair in N'
                remove (N'.Pm, N'.Km) from N'
                insert (N'.Pm, N'.Km) as the first pointer and value in N,
                    by shifting other pointers and values right
                replace K' in parent(N) by N'.Km
            end
        end
        else ... symmetric to the then case ...
    end
end

```

Figure 14.21 Deletion of entry from a B⁺-tree.

In other words, the cost of insertion and deletion operations in terms of I/O operations is proportional to the height of the B⁺-tree, and is therefore low. It is the speed of operation on B⁺-trees that makes them a frequently used index structure in database implementations.

In practice, operations on B⁺-trees result in fewer I/O operations than the worst-case bounds. With fanout of 100, and assuming accesses to leaf nodes are uniformly distributed, the parent of a leaf node is 100 times more likely to get accessed than the leaf node. Conversely, with the same fanout, the total number of nonleaf nodes in a B⁺-tree would be just a little more than 1/100th of the number of leaf nodes. As a result, with memory sizes of several gigabytes being common today, for B⁺-trees that are used frequently, even if the relation is very large it is quite likely that most of the nonleaf nodes are already in the database buffer when they are accessed. Thus, typically only one or two I/O operations are required to perform a lookup. For updates, the probability of a node split occurring is correspondingly very small. Depending on the ordering of inserts, with a fanout of 100, only from 1 in 100 to 1 in 50 insertions will result in a node split, requiring more than one block to be written. As a result, on an average an insert will require just a little more than one I/O operation to write updated blocks.

Although B⁺-trees only guarantee that nodes will be at least half full, if entries are inserted in random order, nodes can be expected to be more than two-thirds full on average. If entries are inserted in sorted order, on the other hand, nodes will be only half full. (We leave it as an exercise to the reader to figure out why nodes would be only half full in the latter case.)

14.3.5 Nonunique Search Keys

We have assumed so far that search keys are unique. Recall also that we described earlier, in Section 14.3.1, how to make search keys unique by creating a composite search key containing the original search key and extra attributes, that together are unique across all records.

The extra attribute can be a record-id, which is a pointer to the record, or a primary key, or any other attribute whose value is unique among all records with the same search-key value. The extra attribute is called a **uniquifier** attribute.

A search with the original search-key attribute can be carried out using a range search as we saw in Section 14.3.2; alternatively, we can create a variant of the *findRange* function that takes only the original search key value as parameter and ignores the value of the uniquifier attribute when comparing search-key values.

It is also possible to modify the B⁺-tree structure to support duplicate search keys. The insert, delete, and lookup methods all have to be modified correspondingly.

- One alternative is to store each key value only once in the tree, and to keep a bucket (or list) of record pointers with a search-key value, to handle nonunique search keys. This approach is space efficient since it stores the key value only once; however, it creates several complications when B⁺-trees are implemented. If the

buckets are kept in the leaf node, extra code is needed to deal with variable-size buckets, and to deal with buckets that grow larger than the size of the leaf node. If the buckets are stored in separate blocks, an extra I/O operation may be required to fetch records.

- Another option is to store the search key value once per record; this approach allows a leaf node to be split in the usual way if it is found to be full during an insert. However, this approach makes handling of split and search on internal nodes significantly more complicated, since two leaves may contain the same search key value. It also has a higher space overhead, since key values are stored as many times as there are records containing that value.

A major problem with both these approaches, as compared to the unique search-key approach, lies in the efficiency of record deletion. (The complexity of lookup and insertion are the same with both these approaches, as well as with the unique search-key approach.) Suppose a particular search-key value occurs a large number of times, and one of the records with that search key is to be deleted. The deletion may have to search through a number of entries with the same search-key value, potentially across multiple leaf nodes, to find the entry corresponding to the particular record being deleted. Thus, the worst-case complexity of deletion may be linear in the number of records.

In contrast, record deletion can be done efficiently using the unique search key approach. When a record is to be deleted, the composite search-key value is computed from the record and then used to look up the index. Since the value is unique, the corresponding leaf-level entry can be found with a single traversal from root to leaf, with no further accesses at the leaf level. The worst-case cost of deletion is logarithmic in the number of records, as we saw earlier.

Due to the inefficiency of deletion, as well as other complications due to duplicate search keys, B⁺-tree implementations in most database systems only handle unique search keys, and they automatically add record-ids or other attributes to make nonunique search keys unique.

14.4 B⁺-Tree Extensions

In this section, we discuss several extensions and variations of the B⁺-tree index structure.

14.4.1 B⁺-Tree File Organization

As mentioned in Section 14.3, the main drawback of index-sequential file organization is the degradation of performance as the file grows: With growth, an increasing percentage of index entries and actual records become out of order and are stored in overflow blocks. We solve the degradation of index lookups by using B⁺-tree indices on the file.

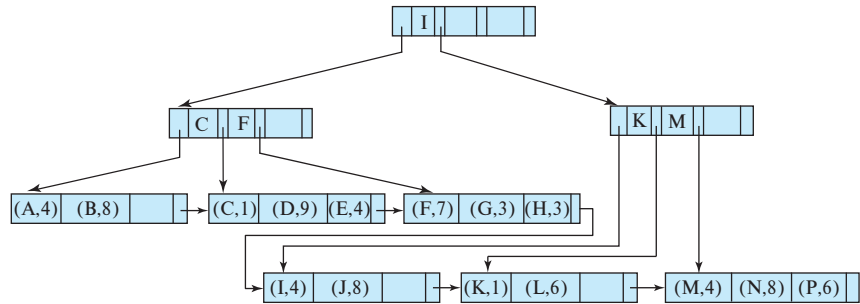


Figure 14.22 B⁺-tree file organization.

We solve the degradation problem for storing the actual records by using the leaf level of the B⁺-tree to organize the blocks containing the actual records. We use the B⁺-tree structure not only as an index, but also as an organizer for records in a file. In a **B⁺-tree file organization**, the leaf nodes of the tree store records, instead of storing pointers to records. Figure 14.22 shows an example of a B⁺-tree file organization. Since records are usually larger than pointers, the maximum number of records that can be stored in a leaf node is less than the number of pointers in a nonleaf node. However, the leaf nodes are still required to be at least half full.

Insertion and deletion of records from a B⁺-tree file organization are handled in the same way as insertion and deletion of entries in a B⁺-tree index. When a record with a given key value v is inserted, the system locates the block that should contain the record by searching the B⁺-tree for the largest key in the tree that is $\leq v$. If the block located has enough free space for the record, the system stores the record in the block. Otherwise, as in B⁺-tree insertion, the system splits the block in two and redistributes the records in it (in the B⁺-tree-key order) to create space for the new record. The split propagates up the B⁺-tree in the normal fashion. When we delete a record, the system first removes it from the block containing it. If a block B becomes less than half full as a result, the records in B are redistributed with the records in an adjacent block B' . Assuming fixed-sized records, each block will hold at least one-half as many records as the maximum that it can hold. The system updates the nonleaf nodes of the B⁺-tree in the usual fashion.

When we use a B⁺-tree for file organization, space utilization is particularly important, since the space occupied by the records is likely to be much more than the space occupied by keys and pointers. We can improve the utilization of space in a B⁺-tree by involving more sibling nodes in redistribution during splits and merges. The technique is applicable to both leaf nodes and nonleaf nodes, and it works as follows:

During insertion, if a node is full, the system attempts to redistribute some of its entries to one of the adjacent nodes, to make space for a new entry. If this attempt fails because the adjacent nodes are themselves full, the system splits the node and divides the entries evenly among one of the adjacent nodes and the two nodes that it obtained by splitting the original node. Since the three nodes together contain one more record

than can fit in two nodes, each node will be about two-thirds full. More precisely, each node will have at least $\lfloor 2n/3 \rfloor$ entries, where n is the maximum number of entries that the node can hold. ($\lfloor x \rfloor$ denotes the greatest integer that is less than or equal to x ; that is, we drop the fractional part, if any.)

During deletion of a record, if the occupancy of a node falls below $\lfloor 2n/3 \rfloor$, the system attempts to borrow an entry from one of the sibling nodes. If both sibling nodes have $\lfloor 2n/3 \rfloor$ records, instead of borrowing an entry, the system redistributes the entries in the node and in the two siblings evenly between two of the nodes and deletes the third node. We can use this approach because the total number of entries is $3\lfloor 2n/3 \rfloor - 1$, which is less than $2n$. With three adjacent nodes used for redistribution, each node can be guaranteed to have $\lfloor 3n/4 \rfloor$ entries. In general, if m nodes ($m-1$ siblings) are involved in redistribution, each node can be guaranteed to contain at least $\lfloor (m-1)n/m \rfloor$ entries. However, the cost of update becomes higher as more sibling nodes are involved in the redistribution.

Note that in a B⁺-tree index or file organization, leaf nodes that are adjacent to each other in the tree may be located at different places on disk. When a file organization is newly created on a set of records, it is possible to allocate blocks that are mostly contiguous on disk to leaf nodes that are contiguous in the tree. Thus, a sequential scan of leaf nodes would correspond to a mostly sequential scan on disk. As insertions and deletions occur on the tree, sequentiality is increasingly lost, and sequential access has to wait for disk seeks increasingly often. An index rebuild may be required to restore sequentiality.

B⁺-tree file organizations can also be used to store large objects, such as SQL clobs and blobs, which may be larger than a disk block, and as large as multiple gigabytes. Such large objects can be stored by splitting them into sequences of smaller records that are organized in a B⁺-tree file organization. The records can be sequentially numbered, or numbered by the byte offset of the record within the large object, and the record number can be used as the search key.

14.4.2 Secondary Indices and Record Relocation

Some file organizations, such as the B⁺-tree file organization, may change the location of records even when the records have not been updated. As an example, when a leaf node is split in a B⁺-tree file organization, a number of records are moved to a new node. In such cases, all secondary indices that store pointers to the relocated records would have to be updated, even though the values in the records may not have changed. Each leaf node may contain a fairly large number of records, and each of them may be in different locations on each secondary index. Thus, a leaf-node split may require tens or even hundreds of I/O operations to update all affected secondary indices, making it a very expensive operation.

A widely used solution for this problem is as follows: In secondary indices, in place of pointers to the indexed records, we store the values of the primary-index search-key

attributes. For example, suppose we have a primary index on the attribute *ID* of relation *instructor*; then a secondary index on *dept_name* would store with each department name a list of instructor's *ID* values of the corresponding records, instead of storing pointers to the records.

Relocation of records because of leaf-node splits then does not require any update on any such secondary index. However, locating a record using the secondary index now requires two steps: First we use the secondary index to find the primary-index search-key values, and then we use the primary index to find the corresponding records.

This approach thus greatly reduces the cost of index update due to file reorganization, although it increases the cost of accessing data using a secondary index.

14.4.3 Indexing Strings

Creating B⁺-tree indices on string-valued attributes raises two problems. The first problem is that strings can be of variable length. The second problem is that strings can be long, leading to a low fanout and a correspondingly increased tree height.

With variable-length search keys, different nodes can have different fanouts even if they are full. A node must then be split if it is full, that is, there is no space to add a new entry, regardless of how many search entries it has. Similarly, nodes can be merged or entries redistributed depending on what fraction of the space in the nodes is used, instead of being based on the maximum number of entries that the node can hold.

The fanout of nodes can be increased by using a technique called **prefix compression**. With prefix compression, we do not store the entire search key value at nonleaf nodes. We only store a prefix of each search key value that is sufficient to distinguish between the key values in the subtrees that it separates. For example, if we had an index on names, the key value at a nonleaf node could be a prefix of a name; it may suffice to store “Silb” at a nonleaf node, instead of the full “Silberschatz” if the closest values in the two subtrees that it separates are, say, “Silas” and “Silver” respectively.

14.4.4 Bulk Loading of B⁺-Tree Indices

As we saw earlier, insertion of a record in a B⁺-tree requires a number of I/O operations that in the worst case is proportional to the height of the tree, which is usually fairly small (typically five or less, even for large relations).

Now consider the case where a B⁺-tree is being built on a large relation. Suppose the relation is significantly larger than main memory, and we are constructing a non-clustering index on the relation such that the index is also larger than main memory. In this case, as we scan the relation and add entries to the B⁺-tree, it is quite likely that each leaf node accessed is not in the database buffer when it is accessed, since there is no particular ordering of the entries. With such randomly ordered accesses to blocks, each time an entry is added to the leaf, a disk seek will be required to fetch the block containing the leaf node. The block will probably be evicted from the disk buffer before another entry is added to the block, leading to another disk seek to write the block back

to disk. Thus, a random read and a random write operation may be required for each entry inserted.

For example, if the relation has 100 million records, and each I/O operation takes about 10 milliseconds on a magnetic disk, it would take at least 1 million seconds to build the index, counting only the cost of reading leaf nodes, not even counting the cost of writing the updated nodes back to disk. This is clearly a very large amount of time; in contrast, if each record occupies 100 bytes, and the disk subsystem can transfer data at 50 megabytes per second, it would take just 200 seconds to read the entire relation.

Insertion of a large number of entries at a time into an index is referred to as **bulk loading** of the index. An efficient way to perform bulk loading of an index is as follows: First, create a temporary file containing index entries for the relation, then sort the file on the search key of the index being constructed, and finally scan the sorted file and insert the entries into the index. There are efficient algorithms for sorting large relations, described later in Section 15.4, which can sort even a large file with an I/O cost comparable to that of reading the file a few times, assuming a reasonable amount of main memory is available.

There is a significant benefit to sorting the entries before inserting them into the B^+ -tree. When the entries are inserted in sorted order, all entries that go to a particular leaf node will appear consecutively, and the leaf needs to be written out only once; nodes will never have to be read from disk during bulk load, if the B^+ -tree was empty to start with. Each leaf node will thus incur only one I/O operation even though many entries may be inserted into the node. If each leaf contains 100 entries, the leaf level will contain 1 million nodes, resulting in only 1 million I/O operations for creating the leaf level. Even these I/O operations can be expected to be sequential, if successive leaf nodes are allocated on successive disk blocks, and few disk seeks would be required. With magnetic disks, 1 millisecond per block is a reasonable estimate for mostly sequential I/O operations, in contrast to 10 milliseconds per block for random I/O operations.

We shall study the cost of sorting a large relation later, in Section 15.4, but as a rough estimate, the index which would have otherwise taken up to 1,000,000 seconds to build on a magnetic disk can be constructed in well under 1000 seconds by sorting the entries before inserting them into the B^+ -tree.

If the B^+ -tree is initially empty, it can be constructed faster by building it bottom-up, from the leaf level, instead of using the usual insert procedure. In **bottom-up B^+ -tree construction**, after sorting the entries as we just described, we break up the sorted entries into blocks, keeping as many entries in a block as can fit in the block; the resulting blocks form the leaf level of the B^+ -tree. The minimum value in each block, along with the pointer to the block, is used to create entries in the next level of the B^+ -tree, pointing to the leaf blocks. Each further level of the tree is similarly constructed using the minimum values associated with each node one level below, until the root is created. We leave details as an exercise for the reader.

Most database systems implement efficient techniques based on sorting of entries, and bottom-up construction, when creating an index on a relation, although they use

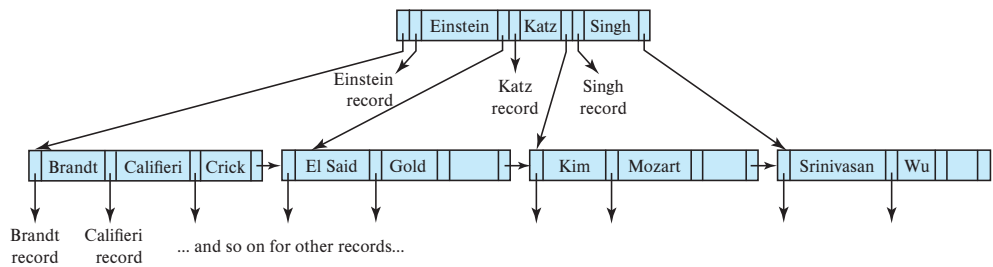


Figure 14.23 B-tree equivalent of B⁺-tree in Figure 14.9.

the normal insertion procedure when tuples are added one at a time to a relation with an existing index. Some database systems recommend that if a very large number of tuples are added at once to an already existing relation, indices on the relation (other than any index on the primary key) should be dropped, and then re-created after the tuples are inserted, to take advantage of efficient bulk-loading techniques.

14.4.5 B-Tree Index Files

B-tree indices are similar to B⁺-tree indices. The primary distinction between the two approaches is that a B-tree eliminates the redundant storage of search-key values. In the B⁺-tree of Figure 14.9, the search keys “Einstein”, “Gold”, “Mozart”, and “Srinivasan” appear in nonleaf nodes, in addition to appearing in the leaf nodes. Every search-key value appears in some leaf node; several are repeated in nonleaf nodes.

A B-tree allows search-key values to appear only once (if they are unique), unlike a B⁺-tree, where a value may appear in a nonleaf node, in addition to appearing in a leaf node. Figure 14.23 shows a B-tree that represents the same search keys as the B⁺-tree of Figure 14.9. Since search keys are not repeated in the B-tree, we may be able to store the index in fewer tree nodes than in the corresponding B⁺-tree index. However, since search keys that appear in nonleaf nodes appear nowhere else in the B-tree, we are forced to include an additional pointer field for each search key in a nonleaf node. These additional pointers point to either file records or buckets for the associated search key.

It is worth noting that many database system manuals, articles in industry literature, and industry professionals use the term **B-tree** to refer to the data structure that we call the B⁺-tree. In fact, it would be fair to say that in current usage, the term **B-tree** is assumed to be synonymous with B⁺-tree. However, in this book we use the terms **B-tree** and B⁺-tree as they were originally defined, to avoid confusion between the two data structures.

A generalized B-tree leaf node appears in Figure 14.24a; a nonleaf node appears in Figure 14.24b. Leaf nodes are the same as in B⁺-trees. In nonleaf nodes, the pointers P_i are the tree pointers that we used also for B⁺-trees, while the pointers B_i are bucket or file-record pointers. In the generalized B-tree in the figure, there are $n - 1$ keys in

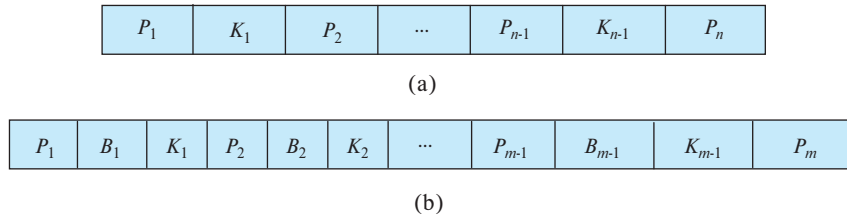


Figure 14.24 Typical nodes of a B-tree. (a) Leaf node. (b) Nonleaf node.

the leaf node, but there are $m - 1$ keys in the nonleaf node. This discrepancy occurs because nonleaf nodes must include pointers B_i , thus reducing the number of search keys that can be held in these nodes. Clearly, $m < n$, but the exact relationship between m and n depends on the relative size of search keys and pointers.

The number of nodes accessed in a lookup in a B-tree depends on where the search key is located. A lookup on a B^+ -tree requires traversal of a path from the root of the tree to some leaf node. In contrast, it is sometimes possible to find the desired value in a B-tree before reaching a leaf node. However, roughly n times as many keys are stored in the leaf level of a B-tree as in the nonleaf levels, and, since n is typically large, the benefit of finding certain values early is relatively small. Moreover, the fact that fewer search keys appear in a nonleaf B-tree node, compared to B^+ -trees, implies that a B-tree has a smaller fanout and therefore may have depth greater than that of the corresponding B^+ -tree. Thus, lookup in a B-tree is faster for some search keys but slower for others, although, in general, lookup time is still proportional to the logarithm of the number of search keys.

Deletion in a B-tree is more complicated. In a B^+ -tree, the deleted entry always appears in a leaf. In a B-tree, the deleted entry may appear in a nonleaf node. The proper value must be selected as a replacement from the subtree of the node containing the deleted entry. Specifically, if search key K_i is deleted, the smallest search key appearing in the subtree of pointer P_{i+1} must be moved to the field formerly occupied by K_i . Further actions need to be taken if the leaf node now has too few entries. In contrast, insertion in a B-tree is only slightly more complicated than is insertion in a B^+ -tree.

The space advantages of B-trees are marginal for large indices and usually do not outweigh the disadvantages that we have noted. Thus, pretty much all database-system implementations use the B^+ -tree data structure, even if (as we discussed earlier) they refer to the data structure as a B-tree.

14.4.6 Indexing on Flash Storage

In our description of indexing so far, we have assumed that data are resident on magnetic disks. Although this assumption continues to be true for the most part, flash storage capacities have grown significantly, and the cost of flash storage per gigabyte has dropped correspondingly, and flash based SSD storage has now replaced magnetic-disk storage for many applications.

Standard B⁺-tree indices can continue to be used even on SSDs, with acceptable update performance and significantly improved lookup performance compared to disk storage.

Flash storage is structured as pages, and the B⁺-tree index structure can be used with flash based SSDs. SSDs provide much faster random I/O operations than magnetic disks, requiring only around 20 to 100 microseconds for a random page read, instead of about 5 to 10 milliseconds with magnetic disks. Thus, lookups run much faster with data on SSDs, compared to data on magnetic disks.

The performance of write operations is more complicated with flash storage. An important difference between flash storage and magnetic disks is that flash storage does not permit in-place updates to data at the physical level, although it appears to do so logically. Every update turns into a copy+write of an entire flash-storage page, requiring the old copy of the page to be erased subsequently. A new page can be written in 20 to 100 microseconds, but eventually old pages need to be erased to free up the pages for further writes. Erases are done at the level of blocks containing multiple pages, and a block erase takes 2 to 5 milliseconds.

The optimum B⁺-tree node size for flash storage is smaller than that with magnetic disk, since flash pages are smaller than disk blocks; it makes sense for tree-node sizes to match to flash pages, since larger nodes would lead to multiple page writes when a node is updated. Although smaller pages lead to taller trees and more I/O operations to access data, random page reads are so much faster with flash storage that the overall impact on read performance is quite small.

Although random I/O is much cheaper with SSDs than with magnetic disks, bulk loading still provides significant performance benefits, compared to tuple-at-a-time insertion, with SSDs. In particular, bottom-up construction reduces the number of page writes compared to tuple-at-a-time insertion, even if the entries are sorted on the search key. Since page writes on flash cannot be done in place and require relatively expensive block erases at a later point in time, the reduction of number of page writes with bottom-up B⁺-tree construction provides significant performance benefits.

Several extensions and alternatives to B⁺-trees have been proposed for flash storage, with a focus on reducing the number of erase operations that result due to page rewrites. One approach is to add buffers to internal nodes of B⁺-trees and record updates temporarily in buffers at higher levels, pushing the updates down to lower levels lazily. The key idea is that when a page is updated, multiple updates are applied together, reducing the number of page writes per update. Another approach creates multiple trees and merges them; the log-structured merge tree and its variants are based on this idea. In fact, both these approaches are also useful for reducing the cost of writes on magnetic disks; we outline both these approaches in Section 14.8.

14.4.7 Indexing in Main Memory

Main memory today is large and cheap enough that many organizations can afford to buy enough main memory to fit all their operational data in-memory. B⁺-trees can

CHAPTER 24



Advanced Indexing Techniques

We studied the concept of indexing, as well as a number of different index structures in Chapter 14. While some index structures, such as B⁺-trees, were covered in detail, others such as hashing, write-optimized indices, bitmap indices, and spatial indices were only briefly outlined in Chapter 14. In this chapter we provide further details of these index structures. We provide detailed coverage of the LSM tree and its variants. We then provide a detailed description of bitmap indices. Next, we provide more detailed coverage of spatial indexing, covering quad trees and R-trees in more detail. Finally, we cover hashing, with detailed coverage of dynamic hashing techniques.

24.1 Bloom Filter

A **Bloom filter** is a probabilistic data structure that can check for membership of a value in a set using very little space, but at a small risk of overestimating the set of elements that are in the set. A Bloom filter is basically a bitmap. If the set has n values, the associated bitmap has a few times n (typically $10n$) bits; the Bloom filter also has associated with it several hash functions. We assume initially that there is only one hash function $h()$.

The bits in the bitmap are all initially set to 0; subsequently, each value in the set is read, and the hash function $h(v)$ is computed on the element v , with the range of the function being 1 to $10n$. The bit at position $h(v)$ is then set to 1. This is repeated for every element v . To check if a particular value v is present in the set, the hash function $h(v)$ is computed. If bit $h(v)$ in the Bloom filter is equal to 0, we can infer that v cannot possibly be in the set. However, if bit $h(v)$ is equal to 1, v may be present in the set. Note that with some probability, the bit $h(v)$ may be 1 even if v is not present, if some other value v' , present in the set has $h(v') = h(v)$. Thus, a lookup for v results in a *false positive*.

To reduce the chance of false positives, Bloom filters use k independent hash functions $h_i()$, $i = 1..k$, for some $k > 1$; for each value v in the set, bits corresponding to $h_i(v)$, $i = 1..k$ are all set to 1 in the bitmap. When querying the Bloom filter with a

24.2.5 LSM Trees For Flash Storage

LSM trees were initially designed to reduce the write and seek overheads of hard disks. Flash disks have a relatively low overhead for random I/O operations since they do not require seek, and thus the benefit of avoiding random I/O that LSM tree variants provide is not particularly important with flash disks.

However, recall that flash memory does not allow in-place update, and writing even a single byte to a page requires the whole page to be rewritten to a new physical location; the original location of the page needs to be erased eventually, which is a relatively expensive operation. The reduction in write amplification using LSM tree variants, as compared to traditional B⁺-trees, can provide substantial performance benefits when LSM trees are used with flash storage.

24.3 Bitmap Indices

As we saw in Section 14.9, a bitmap index is a specialized type of index designed for easy querying on multiple keys. Bitmaps work best for attributes that take only a small number of distinct values.

For bitmap indices to be used, records in a relation must be numbered sequentially, starting from, say, 0. Given a number n , it must be easy to retrieve the record numbered n . This is particularly easy to achieve if records are fixed in size and allocated on consecutive blocks of a file. The record number can then be translated easily into a block number and a number that identifies the record within the block.

Recall that *column-oriented storage*, described in Section 13.6, stores attributes in arrays, allowing efficient access of the attribute of the i th record, for any given i . Bitmap indices are thus particularly useful with columnar storage.

We use as a running example a relation *instructor_info*, which has an attribute *gender*, which can take only values **m** (male) or **f** (female), and an attribute *income_level*, where income has been broken up into 5 levels: $L1$: 0-9999, $L2$: 10,000-19,999, $L3$: 20,000-39,999, $L4$: 40,000-74,999, and $L5$: 75,000 – ∞ .

24.3.1 Bitmap Index Structure

As we saw in Section 14.9, a **bitmap** is simply an array of bits. In its simplest form, a **bitmap index** on the attribute A of relation r consists of one bitmap for each value that A can take. Each bitmap has as many bits as the number of records in the relation. The i th bit of the bitmap for value v_j is set to 1 if the record numbered i has the value v_j for attribute A . All other bits of the bitmap are set to 0.

In our example, there is one bitmap for the value **m** and one for **f**. The i th bit of the bitmap for **m** is set to 1 if the *gender* value of the record numbered i is **m**. All other bits of the bitmap for **m** are set to 0. Similarly, the bitmap for **f** has the value 1 for bits corresponding to records with the value **f** for the *gender* attribute; all other bits have

record number				Bitmaps for <i>gender</i>		Bitmaps for <i>income_level</i>	
	<i>ID</i>	<i>gender</i>	<i>income_level</i>	m	f		
0	76766	m	L1		10010	L1	10100
1	22222	f	L2		01101	L2	01000
2	12121	f	L1			L3	00001
3	15151	m	L4			L4	00010
4	58583	f	L3			L5	00000

Figure 24.3 Bitmap indices on relation *instructor_info*.

the value 0. Figure 24.3 shows an example of bitmap indices on a relation *instructor_info*.

We now consider when bitmaps are useful. The simplest way of retrieving all records with value *m* (or value *f*) would be to simply read all records of the relation and select those records with value *m* (or *f*, respectively). The bitmap index doesn't really help to speed up such a selection. While it would allow us to read only those records for a specific gender, it is likely that every disk block for the file would have to be read anyway.

In fact, bitmap indices are useful for selections mainly when there are selections on multiple keys. Suppose we create a bitmap index on attribute *income_level*, which we described earlier, in addition to the bitmap index on *gender*.

Consider now a query that selects women with income in the range \$10,000 to \$19,999. This query can be expressed as

```
select *
from instructor_info
where gender = 'f' and income_level = 'L2';
```

To evaluate this selection, we fetch the bitmaps for *gender* value *f* and the bitmap for *income_level* value *L2* and perform an **intersection** (logical-and) of the two bitmaps. In other words, we compute a new bitmap where bit *i* has value 1 if the *i*th bit of the two bitmaps are both 1 and has a value 0 otherwise. In the example in Figure 24.3, the intersection of the bitmap for *gender* = *f* (01101) and the bitmap for *income_level* = *L2* (01000) gives the bitmap 01000.

Since the first attribute can take two values, and the second can take five values, we would expect only about 1 in 10 records, on an average, to satisfy a combined condition on the two attributes. If there are further conditions, the fraction of records satisfying all the conditions is likely to be quite small. The system can then compute the query result by finding all bits with value 1 in the intersection bitmap and retrieving the cor-

responding records. If the fraction is large, scanning the entire relation would remain the cheaper alternative.

Another important use of bitmaps is to count the number of tuples satisfying a given selection. Such queries are important for data analysis. For instance, if we wish to find out how many women have an income level $L2$, we compute the intersection of the two bitmaps and then count the number of bits that are 1 in the intersection bitmap. We can thus get the desired result from the bitmap index, without even accessing the relation.

Bitmap indices are generally quite small compared to the actual relation size. Records are typically at least tens of bytes to hundreds of bytes long, whereas a single bit represents the record in a bitmap. Thus, the space occupied by a single bitmap is usually less than 1 percent of the space occupied by the relation. For instance, if the record size for a given relation is 100 bytes, then the space occupied by a single bitmap will be $\frac{1}{8}$ of 1 percent of the space occupied by the relation. If an attribute A of the relation can take on only one of eight values, a bitmap index on attribute A would consist of eight bitmaps, which together occupy only 1 percent of the size of the relation.

Deletion of records creates gaps in the sequence of records, since shifting records (or record numbers) to fill gaps would be extremely expensive. To recognize deleted records, we can store an **existence bitmap**, in which bit i is 0 if record i does not exist and 1 otherwise. We shall see the need for existence bitmaps in Section 24.3.2. Insertion of records should not affect the sequence numbering of other records. Therefore, we can do insertion either by appending records to the end of the file or by replacing deleted records.

24.3.2 Efficient Implementation of Bitmap Operations

We can compute the intersection of two bitmaps easily by using a **for** loop: the i th iteration of the loop computes the **and** of the i th bits of the two bitmaps. We can speed up computation of the intersection greatly by using bit-wise **and** instructions supported by most computer instruction sets. A *word* usually consists of 32 or 64 bits, depending on the architecture of the computer. A bit-wise **and** instruction takes two words as input and outputs a word where each bit is the logical **and** of the bits in corresponding positions of the input words. What is important to note is that a single bit-wise **and** instruction can compute the intersection of 32 or 64 bits *at once*.

If a relation had 1 million records, each bitmap would contain 1 million bits, or equivalently 128 kilobytes. Only 31,250 instructions are needed to compute the intersection of two bitmaps for our relation, assuming a 32-bit word length. Thus, computing bitmap intersections is an extremely fast operation.

Just as bitmap intersection is useful for computing the **and** of two conditions, bitmap union is useful for computing the **or** of two conditions. The procedure for bitmap union is exactly the same as for intersection, except we use bit-wise **or** instructions instead of bit-wise **and** instructions.

The complement operation can be used to compute a predicate involving the negation of a condition, such as **not** (*income_level* = *L1*). The complement of a bitmap is generated by complementing every bit of the bitmap (the complement of 1 is 0 and the complement of 0 is 1). It may appear that **not** (*income_level* = *L1*) can be implemented by just computing the complement of the bitmap for income level *L1*. If some records have been deleted, however, just computing the complement of a bitmap is not sufficient. Bits corresponding to such records would be 0 in the original bitmap but would become 1 in the complement, although the records don't exist. A similar problem also arises when the value of an attribute is *null*. For instance, if the value of *income_level* is null, the bit would be 0 in the original bitmap for value *L1* and 1 in the complement bitmap.

To make sure that the bits corresponding to deleted records are set to 0 in the result, the complement bitmap must be intersected with the existence bitmap to turn off the bits for deleted records. Similarly, to handle null values, the complement bitmap must also be intersected with the complement of the bitmap for the value *null*.¹

Counting the number of bits that are 1 in a bitmap can be done quickly by a clever technique. We can maintain an array with 256 entries, where the *i*th entry stores the number of bits that are 1 in the binary representation of *i*. Set the total count initially to 0. We take each byte of the bitmap, use it to index into this array, and add the stored count to the total count. The number of addition operations is $\frac{1}{8}$ of the number of tuples, and thus the counting process is very efficient. A large array (using $2^{16} = 65,536$ entries), indexed by pairs of bytes, would give even higher speedup, but at a higher storage cost.

24.3.3 Bitmaps and B⁺-Trees

Bitmaps can be combined with regular B⁺-tree indices for relations where a few attribute values are extremely common, and other values also occur, but much less frequently. In a B⁺-tree index leaf, for each value we would normally maintain a list of all records with that value for the indexed attribute. Each element of the list would be a record identifier, consisting of at least 32 bits, and usually more. For a value that occurs in many records, we store a bitmap instead of a list of records.

Suppose a particular value v_i occurs in $\frac{1}{16}$ of the records of a relation. Let N be the number of records in the relation, and assume that a record has a 64-bit number identifying it. The bitmap needs only 1 bit per record, or N bits in total. In contrast, the list representation requires 64 bits per record where the value occurs, or $64 * N/16 = 4N$ bits. Thus, a bitmap is preferable for representing the list of records for value v_i . In our example (with a 64-bit record identifier), if fewer than 1 in 64 records have a particular value, the list representation is preferable for identifying records with that

¹Handling predicates such as **is unknown** would cause further complications, which would in general require use of an extra bitmap to track which operation results are unknown.

value, since it uses fewer bits than the bitmap representation. If more than 1 in 64 records have that value, the bitmap representation is preferable.

Thus, bitmaps can be used as a compressed storage mechanism at the leaf nodes of B⁺-trees for those values that occur very frequently.

24.4 Indexing of Spatial Data

As we saw in Section 14.10.1, indices are required for efficient access to spatial data, and such indices must efficiently support queries such as range and nearest neighbor queries. We also gave a brief overview of k-d trees, quadrees, and R-trees; we also briefly described how to answer range queries using k-d trees. In this section we provide further details of quadrees and R-trees.

As mentioned in Section 14.10.1, in addition to indexing of points, spatial indices must also support indexing of regions of space such as line segments, rectangles, and other polygons. There are extensions of k-d trees and quadrees for this task. However, a line segment or polygon may cross a partitioning line. If it does, it has to be split and represented in each of the subtrees in which its pieces occur. Multiple occurrences of a line segment or polygon caused by such splits can result in inefficiencies in storage, as well as inefficiencies in querying. R-trees were developed to support efficient indexing of such structures.

24.4.1 Quadrees

An alternative representation for two-dimensional data are a **quadtree**. An example of the division of space by a quadtree appears in Figure 24.4. Each node of a quadtree is

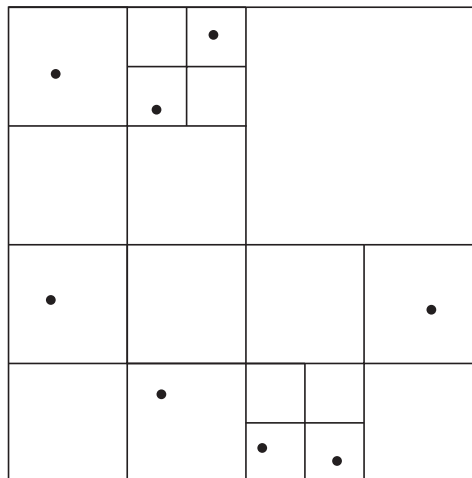


Figure 24.4 Division of space by a quadtree.

associated with a rectangular region of space. The top node is associated with the entire target space. Each nonleaf node in a quadtree divides its region into four equal-sized quadrants, and correspondingly each such node has four child nodes corresponding to the four quadrants. Leaf nodes have between zero and some fixed maximum number of points. Correspondingly, if the region corresponding to a node has more than the maximum number of points, child nodes are created for that node. In the example in Figure 24.4, the maximum number of points in a leaf node is set to 1.

This type of quadtree is called a **PR quadtree**, to indicate it stores points, and that the division of space is divided based on regions, rather than on the actual set of points stored. We can use **region quadtrees** to store array (raster) information. A node in a region quadtree is a leaf node if all the array values in the region that it covers are the same. Otherwise, it is subdivided further into four children of equal area and is therefore an internal node. Each node in the region quadtree corresponds to a subarray of values. The subarrays corresponding to leaves either contain just a single array element or have multiple array elements, all of which have the same value.

24.4.2 R-Trees

A storage structure called an **R-tree** is useful for indexing of objects such as points, line segments, rectangles, and other polygons. An R-tree is a balanced tree structure with the indexed objects stored in leaf nodes, much like a B⁺-tree. However, instead of a range of values, a rectangular **bounding box** is associated with each tree node. The bounding box of a leaf node is the smallest rectangle parallel to the axes that contains all objects stored in the leaf node. The bounding box of internal nodes is, similarly, the smallest rectangle parallel to the axes that contains the bounding boxes of its child nodes. The bounding box of an object (such as a polygon) is defined, similarly, as the smallest rectangle parallel to the axes that contains the object.

Each internal node stores the bounding boxes of the child nodes along with the pointers to the child nodes. Each leaf node stores the indexed objects and may optionally store the bounding boxes of the objects; the bounding boxes help speed up checks for overlaps of the rectangle with the indexed objects—if a query rectangle does not overlap with the bounding box of an object, it cannot overlap with the object, either. (If the indexed objects are rectangles, there is no need to store bounding boxes, since they are identical to the rectangles.)

Figure 24.5 shows an example of a set of rectangles (drawn with a solid line) and the bounding boxes (drawn with a dashed line) of the nodes of an R-tree for the set of rectangles. Note that the bounding boxes are shown with extra space inside them, to make them stand out pictorially. In reality, the boxes would be smaller and fit tightly on the objects that they contain; that is, each side of a bounding box B would touch at least one of the objects or bounding boxes that are contained in B .

The R-tree itself is at the right side of Figure 24.5. The figure refers to the coordinates of bounding box i as BB_i in the figure.

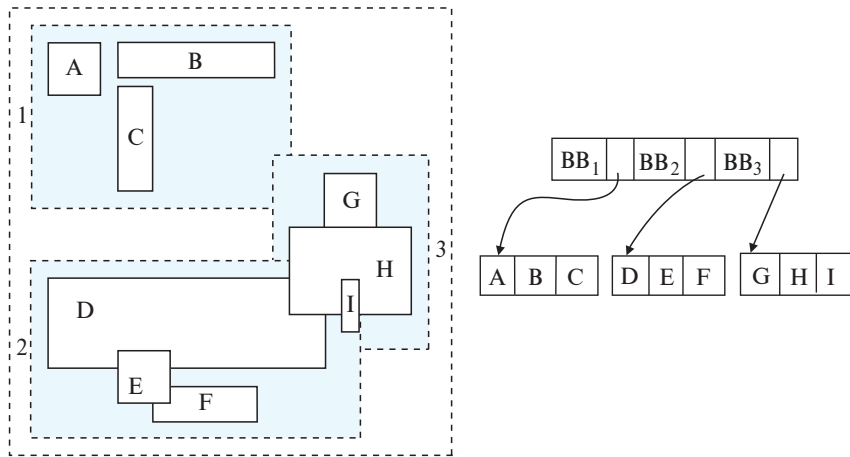


Figure 24.5 An R-tree.

We shall now see how to implement search, insert, and delete operations on an R-tree.

- **Search.** As the figure shows, the bounding boxes associated with sibling nodes may overlap; in B^+ -trees, k-d trees, and quadtrees, in contrast, the ranges do not overlap. A search for objects containing a point therefore has to follow *all* child nodes whose associated bounding boxes contain the point; as a result, multiple paths may have to be searched. Similarly, a query to find all objects that intersect a given object has to go down every node where the associated rectangle intersects the given object.
- **Insert.** When we insert an object into an R-tree, we select a leaf node to hold the object. Ideally we should pick a leaf node that has space to hold a new entry, and whose bounding box contains the bounding box of the object. However, such a node may not exist; even if it did, finding the node may be very expensive, since it is not possible to find it by a single traversal down from the root. At each internal node we may find multiple children whose bounding boxes contain the bounding box of the object, and each of these children needs to be explored. Therefore, as a heuristic, in a traversal from the root, if any of the child nodes has a bounding box containing the bounding box of the object, the R-tree algorithm chooses one of them arbitrarily. If none of the children satisfy this condition, the algorithm chooses a child node whose bounding box has the maximum overlap with the bounding box of the object for continuing the traversal.

Once the leaf node has been reached, if the node is already full, the algorithm performs node splitting (and propagates splitting upward if required) in a manner very similar to B^+ -tree insertion. Just as with B^+ -tree insertion, the R-tree insertion algorithm ensures that the tree remains balanced. Additionally, it ensures that the

bounding boxes of leaf nodes, as well as internal nodes, remain consistent; that is, bounding boxes of leaves contain all the bounding boxes of the objects stored at the leaf, while the bounding boxes for internal nodes contain all the bounding boxes of the children nodes.

The main difference of the insertion procedure from the B^+ -tree insertion procedure lies in how the node is split. In a B^+ -tree, it is possible to find a value such that half the entries are less than the midpoint and half are greater than the value. This property does not generalize beyond one dimension; that is, for more than one dimension, it is not always possible to split the entries into two sets so that their bounding boxes do not overlap. Instead, as a heuristic, the set of entries S can be split into two disjoint sets S_1 and S_2 so that the bounding boxes of S_1 and S_2 have the minimum total area; another heuristic would be to split the entries into two sets S_1 and S_2 in such a way that S_1 and S_2 have minimum overlap. The two nodes resulting from the split would contain the entries in S_1 and S_2 , respectively. The cost of finding splits with minimum total area or overlap can itself be large, so cheaper heuristics, such as the *quadratic split* heuristic, are used. (The heuristic gets its name from the fact that it takes time quadratic in the number of entries.)

The **quadratic split** heuristic works this way: First, it picks a pair of entries a and b from S such that putting them in the same node would result in a bounding box with the maximum wasted space; that is, the area of the minimum bounding box of a and b minus the sum of the areas of a and b is the largest. The heuristic places the entries a and b in sets S_1 and S_2 , respectively.

It then iteratively adds the remaining entries, one entry per iteration, to one of the two sets S_1 or S_2 . At each iteration, for each remaining entry e , let $i_{e,1}$ denote the increase in the size of the bounding box of S_1 if e is added to S_1 and let $i_{e,2}$ denote the corresponding increase for S_2 . In each iteration, the heuristic chooses one of the entries with the maximum difference of $i_{e,1}$ and $i_{e,2}$ and adds it to S_1 if $i_{e,1}$ is less than $i_{e,2}$, and to S_2 otherwise. That is, an entry with “maximum preference” for S_1 or S_2 is chosen at each iteration. The iteration stops when all entries have been assigned, or when one of the sets S_1 or S_2 has enough entries that all remaining entries have to be added to the other set so the nodes constructed from S_1 and S_2 both have the required minimum occupancy. The heuristic then adds all unassigned entries to the set with fewer entries.

- **Deletion.** Deletion can be performed like a B^+ -tree deletion, borrowing entries from sibling nodes, or merging sibling nodes if a node becomes underfull. An alternative approach redistributes all the entries of underfull nodes to sibling nodes, with the aim of improving the clustering of entries in the R-tree.

See the bibliographical references for more details on insertion and deletion operations on R-trees, as well as on variants of R-trees, called R^* -trees or R^+ -trees.

The storage efficiency of R-trees is better than that of k-d trees or quadrees, since an object is stored only once, and we can ensure easily that each node is at least half